

UNIVERSITÀ DEGLI STUDI DI PADOVA
DIPARTIMENTO DI SCIENZE STATISTICHE
CORSO DI LAUREA TRIENNALE IN
STATISTICA PER LE TECNOLOGIE E LE SCIENZE



RELAZIONE FINALE

**Previsione del numero di giocatori attivi per la
scalabilità dei server attraverso il modello Prophet**

Relatore Prof. Denti Francesco
Dipartimento di Scienze Statistiche

Laureando Calori Dakota Davide
Matricola 2067180

Anno Accademico 2024/2025

Abstract

Il gaming online sta vivendo una fase di forte espansione, spinta dalla crescente connettività, dalla diffusione delle piattaforme cross-play e dai progressi nelle tecnologie cloud. In questo contesto, la capacità di scalare efficacemente le infrastrutture server è fondamentale per garantire prestazioni stabili e un'esperienza di gioco fluida, anche durante i picchi di traffico. La gestione ottimale delle risorse rappresenta quindi una sfida strategica per le aziende del settore videoludico.

Questa tesi si concentra sulla previsione del numero di giocatori attivi su Rocket League, uno dei titoli multiplayer più rappresentativi, con l'obiettivo di stimare in modo efficace la quantità di server necessari per soddisfare la domanda, bilanciando costi operativi e qualità del servizio. A tal fine, viene utilizzato un dataset ad alta frequenza che copre il periodo da dicembre 2017 ad agosto 2020, permettendo di analizzare l'evoluzione della domanda in modo dettagliato e su diverse scale temporali.

Viene impiegato il modello Prophet per generare previsioni su orizzonti orari, settimanali e mensili, supportando così decisioni strategiche nel breve e lungo termine. L'analisi considera anche l'impatto di eventi straordinari, come la pandemia di COVID-19, valutando scenari in cui tale shock viene incluso o escluso dal modello predittivo, per comprenderne l'influenza sulla qualità delle stime. I risultati ottenuti con Prophet sono infine confrontati con quelli del modello ARIMA, al fine di analizzare le differenze in termini di accuratezza e adattabilità.

Il lavoro si conclude con una stima del numero ottimale di server da mantenere attivi in funzione della domanda prevista, mostrando come i risultati modellistici possano supportare decisioni informate. L'obiettivo è offrire una base metodologica replicabile per l'ottimizzazione della scalabilità nei contesti di gioco online, contribuendo a una gestione più intelligente ed efficiente delle risorse.

Indice

Introduzione	1
1 Analisi dei dati	3
1.1 Dati	3
1.2 Preprocessing	4
1.3 Analisi Esplorativa	5
1.3.1 Trend generale e statistiche descrittive	5
1.3.2 Analisi giornaliera e oraria	6
1.3.3 Stazionarietà e autocorrelazione	8
2 Il modello Prophet	9
2.1 Verosimiglianza e prior: l'obiettivo del fitting	10
2.2 Trend lineare piecewise	12
2.3 Stagionalità, eventi e festività	13
3 Previsione e confronto tra Prophet e ARIMA	17
3.1 Previsione Oraria	19
3.1.1 Pre-Covid	19
3.1.2 Post-Covid	21
3.2 Previsione Settimanale	24
3.2.1 Pre-Covid	24
3.2.2 Post-Covid	26
3.3 Previsione Mensile	29
3.3.1 Pre-Covid	29
3.3.2 Post-Covid	32
4 Stima della capacità server ottimale	35
4.1 Assunzioni operative	35
4.2 Traduzione del CCU in numero di server	36
5 Conclusioni	39

Introduzione

Nel settore videoludico, comprendere l'andamento del comportamento dei giocatori è essenziale per prendere decisioni informate in ambiti come il game design, la pianificazione degli aggiornamenti e la gestione delle risorse. Con la crescente disponibilità di dati legati all'attività degli utenti, le aziende stanno adottando approcci sempre più guidati dai dati per migliorare il coinvolgimento e la fidelizzazione, dove uno degli indicatori più importanti in questo contesto è il numero di giocatori attivi, che riflette il livello di popolarità e l'engagement di un titolo videoludico. Per i giochi online multiplayer, la rilevanza di tale misura si estende oltre l'aspetto puramente analitico, influenzando direttamente le decisioni infrastrutturali. Il numero di giocatori simultaneamente connessi determina infatti il carico sui server, rendendo cruciale la capacità di dimensionare correttamente le risorse, poiché un'allocazione insufficiente può compromettere la qualità del servizio, mentre un eccesso di capacità comporta costi operativi non necessari. Prevedere con accuratezza la domanda e stimare il numero ottimale di server da mantenere attivi diventa quindi un obiettivo strategico per garantire scalabilità ed efficienza.

Negli ultimi anni, il settore ha visto crescere la complessità gestionale, non solo per l'aumento della base utenti, ma anche per la diffusione di funzionalità cross-platform, che permettono a giocatori provenienti da console e PC diversi di condividere lo stesso ambiente di gioco. A ciò si aggiunge la dimensione competitiva degli e-sport, che porta a picchi di traffico durante i tornei ufficiali e le competizioni online, con impatti infrastrutturali non trascurabili. In questo scenario, la capacità di anticipare tali variazioni non è più soltanto un vantaggio competitivo, ma una necessità per mantenere standard elevati di affidabilità e continuità del servizio.

La presente tesi si concentra sull'analisi e previsione del numero di giocatori attivi su Rocket League, uno dei titoli multiplayer più rappresentativi del panorama videoludico, caratterizzato da un'elevata variabilità della domanda e da un forte legame con eventi

stagionali e promozionali. A partire da un dataset ad alta frequenza che copre il periodo dicembre 2017 – agosto 2020, vengono analizzate le dinamiche della domanda su diverse scale temporali, includendo sia andamenti regolari sia variazioni dovute a eventi straordinari, come la pandemia di COVID-19.

Dal punto di vista metodologico, vengono messi a confronto il modello Prophet, sviluppato da Facebook e particolarmente adatto alla gestione di trend e stagionalità complesse, e ARIMA, un modello statistico classico e ampiamente consolidato nell'analisi delle serie temporali. Questo confronto non si limita a misurare l'accuratezza predittiva, ma ha l'obiettivo più ampio di valutare la capacità dei modelli di fornire indicazioni operative, traducendo le previsioni in stime pratiche sul numero di server da mantenere attivi. Per consentire la replicabilità delle analisi, l'intero codice sviluppato a supporto delle analisi che verranno presentate in questa tesi è reso disponibile pubblicamente su GitHub¹.

L'analisi si inserisce inoltre in una riflessione più ampia sull'uso delle tecniche di previsione applicate alla gestione delle risorse. In un contesto come quello videoludico, caratterizzato da ciclicità, picchi inattesi e shock esogeni, l'adozione di strumenti statistici avanzati può rappresentare un punto d'incontro tra la ricerca accademica e le applicazioni industriali. L'obiettivo finale è quindi duplice: da un lato, contribuire al dibattito metodologico valutando i punti di forza e di debolezza di Prophet e ARIMA; dall'altro, dimostrare come i modelli predittivi possano supportare decisioni concrete in un settore ad alta intensità tecnologica, favorendo una gestione più intelligente ed efficiente delle infrastrutture server.

¹Repository al link: <https://github.com/DakoDC/player-forecasting-with-Prophet-thesis>

Capitolo 1

Analisi dei dati

1.1 Dati

Il dataset analizzato in questa tesi proviene da Select Dataset (2023), una piattaforma che raccoglie e distribuisce dati relativi a diversi ambiti digitali, inclusi i videogiochi online. I dati utilizzati riguardano il gioco *Rocket League*, i quali derivano originariamente dalla Valve Corporation (2025).

Il file contiene 280224 osservazioni ad alta frequenza, registrate ogni 5 minuti, sul numero di giocatori attivi su *Rocket League*. L'intervallo temporale coperto è di quasi tre anni, dal 14 dicembre 2017 al 12 agosto 2020. I dati sono stati raccolti tramite richieste periodiche alle API di Steam, una delle principali piattaforme di distribuzione digitale e multiplayer a livello globale, che fornisce informazioni in tempo reale sull'attività degli utenti per ciascun gioco. Le variabili principali del dataset sono:

- **Time:** timestamp dell'osservazione (UTC);
- **Playercount:** numero di giocatori attivi al momento della rilevazione.

L'elevata granularità consente di effettuare aggregazioni su diverse scale. In questa tesi, oltre alla scala originaria di 5 minuti, verranno considerate anche le scale oraria e giornaliera.

1.2 Preprocessing

Come tipico nei dati ad alta frequenza raccolti tramite API, il dataset presenta alcuni valori distorti o mancanti, dovuti a situazioni come l'interruzione del servizio di gioco, aggiornamenti o la momentanea indisponibilità delle informazioni da parte della piattaforma Steam. L'elaborazione preliminare e le future modellazioni sono state condotte utilizzando il software R, sviluppato da R Core Team (2023), tramite diverse librerie fondamentali per l'analisi delle serie temporali, tra cui `tidyverse`, `prophet`, `tsibble`, `fable` e `forecast`, nonché librerie dedicate alla visualizzazione dei dati, come `ggplot2`, `patchwork` e `gridExtra`.

Per applicare modelli autoregressivi come ARIMA, che richiedono una serie storica completa e regolare, è stato creato un secondo dataset in cui i valori mancanti, corrispondenti a meno dell'1% delle osservazioni, sono stati colmati tramite interpolazione lineare. Questa scelta è stata motivata dalla semplicità del metodo e dalla capacità di preservare il trend generale della serie storica senza introdurre distorsioni nei picchi di attività, fondamentali per l'analisi.

Sono stati inoltre identificati e trattati alcuni valori anomali, ritenuti non plausibili in quanto estremamente bassi rispetto alla distribuzione della serie. In particolare, tutti i valori di `Playercount` inferiori o uguali al quantile 0,0001 sono stati sostituiti con valori mancanti. Tali osservazioni coincidono principalmente con la fine di periodi di downtime dei server, dove quindi sono ancora molto pochi gli accessi al gioco.

Trasformandoli in valori mancanti, sarà possibile sostituirli tramite l'interpolazione con ARIMA, evitando che influiscano negativamente sulle previsioni, portando così i valori mancanti a 2.426 sulle 280.224 osservazioni totali.

Poiché l'obiettivo dell'analisi è stimare con precisione i picchi di attività, i dati sono stati successivamente aggregati su base oraria e giornaliera. Per ogni intervallo è stato mantenuto il valore massimo del numero di giocatori, così da rappresentare i momenti di massimo carico dei server, obiettivo di questa analisi.

1.3 Analisi Esplorativa

Dopo la fase di preprocessing, è stata condotta un'analisi esplorativa dei dati al fine di comprendere l'andamento generale della serie storica ed individuare pattern ricorrenti o eventuali anomalie.

1.3.1 Trend generale e statistiche descrittive

Come accennato prima, il dataset analizzato raccoglie complessivamente 280.224 osservazioni, registrate con una frequenza di cinque minuti. All'interno di questo intervallo di tempo, il numero di giocatori attivi oscilla tra un minimo di circa 4.000 e un picco che supera i 119.000, con una media intorno ai 36.000 giocatori.

Guardando successivamente al grafico della serie giornaliera in Figura 1.1 si evidenzia subito una chiara stagionalità settimanale, dove i valori divisi per giorno della settimana mostrano andamenti simili di crescita e calo, con un aumento sempre più marcato man mano che ci si avvicina al weekend. Prestando invece più attenzione all'andamento della serie, si vede come mostri inizialmente un trend in lieve crescita, che procede in modo piuttosto regolare, ma che col passare del tempo diventa molto più evidente, fino a raggiungere un picco in coincidenza con il periodo della pandemia di COVID-19.

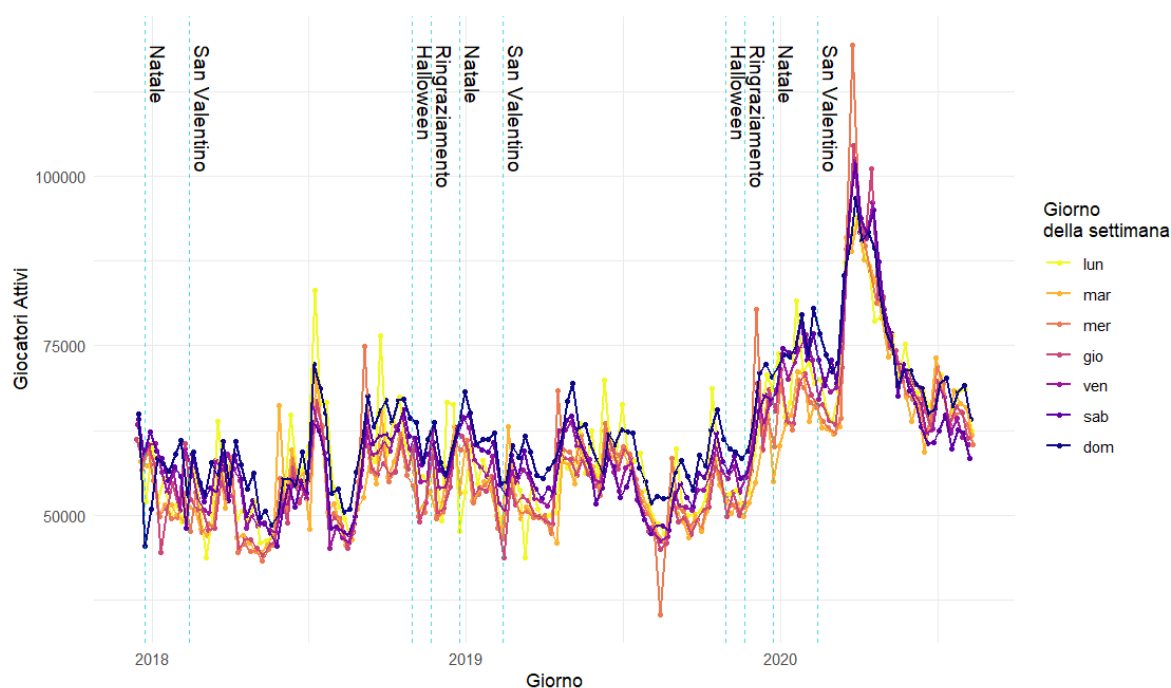


FIGURA 1.1: Andamento del numero massimo giornaliero di giocatori attivi.

Sono inoltre presenti numerosi picchi e cali riconducibili a particolari festività o eventi legati al gioco. Ad esempio, si osserva un aumento significativo di giocatori nei giorni prima e dopo Capodanno, mentre in corrispondenza di San Valentino emerge un calo pronunciato. Questi fenomeni saranno successivamente considerati nel modello **Prophet**, attraverso l'inclusione di specifiche festività e regressori, che permettono a Prophet di imparare a gestire questi eventi al meglio quando si presentano.

1.3.2 Analisi giornaliera e oraria

Analizzando più nello specifico le stagionalità settimanali e giornaliere, emerge che la distribuzione del numero di giocatori segue un andamento piuttosto regolare: durante il fine settimana, in particolare il sabato e la domenica, il numero medio di utenti attivi risulta sensibilmente più alto rispetto ai giorni feriali, con un incremento che si aggira tra il 5% e il 10%, confermando una tendenza abituale, per cui gli utenti tendono a dedicare più tempo al gioco nei giorni non lavorativi.

Il boxplot in Figura 1.2 evidenzia una maggiore variabilità la domenica e nei primi giorni della settimana, in particolare il lunedì. Questo fenomeno si comprende meglio se si pensa alle abitudini della domenica sera, durante la quale molti giocatori smettono prima del solito, consapevoli degli impegni lavorativi del giorno successivo, ma non pochi scelgono di prolungare le sessioni fino a tarda notte. Questo duplice comportamento si riflette in un aumento di variabilità la domenica, ma allo stesso tempo coloro che proseguono fino a notte inoltrata contribuiscono a innalzare le statistiche del lunedì, rendendo questo giorno più variabile e di media maggiore rispetto agli altri giorni feriali.

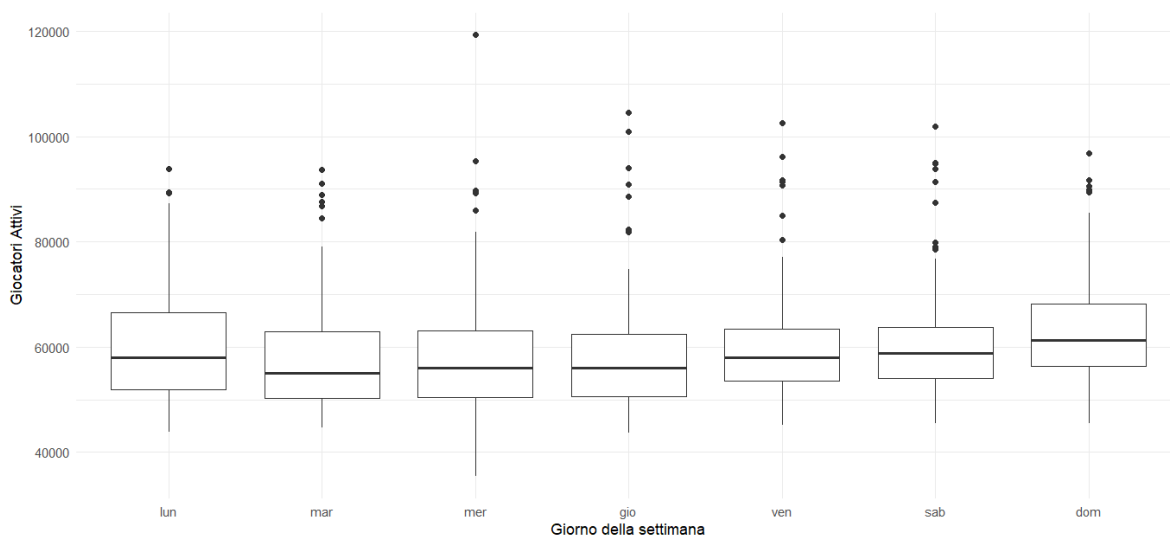


FIGURA 1.2: Distribuzione del numero di giocatori per giorno della settimana.

Successivamente, l'aggregazione oraria ha permesso di esaminare la dinamica giornaliera del numero di giocatori, viene mostrato a tal ragione il boxplot in Figura 1.3, il quale conferma quanto appena detto sull'impatto del weekend; appaiono infatti molteplici valori più elevati rispetto alla media, segno di una maggiore partecipazione nel fine settimana.

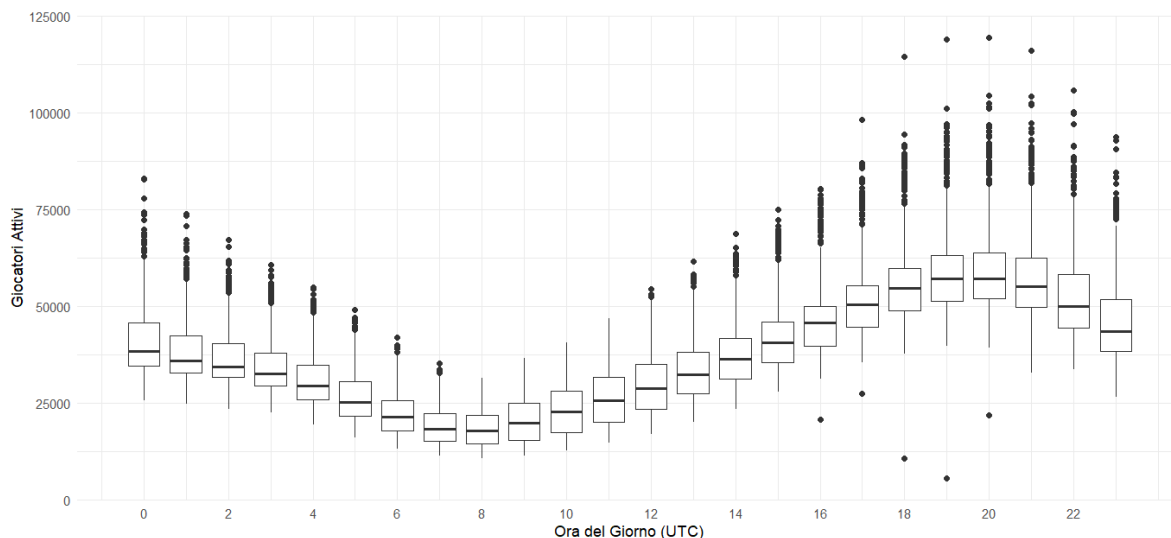


FIGURA 1.3: Giocatori attivi per ora del giorno.

Considerando i fusi orari, va ricordato che l'Europa si colloca tra l'UTC+1 e l'UTC+2 (a seconda dell'ora legale), mentre l'America varia dall'UTC-8 all'UTC-5. Queste due aree geografiche appaiono infatti essere le principali responsabili delle variazioni osservate nel numero di giocatori attivi. L'andamento evidenziato dal grafico risulta infatti coerente con quanto riportato anche dall'applicazione ByteBrew (2025), che ha condotto un'analisi dettagliata sui tempi di gioco per area geografica e secondo cui sia in Europa che negli Stati Uniti, la fascia serale rappresenta il momento di massima attività, con un picco intorno alle ore 20:00 del rispettivo fuso orario, mostra poi come il pomeriggio registri valori solo leggermente inferiori, mentre la mattina corrisponde al periodo di maggior calo dell'attività.

Tutto ciò emerge dal grafico notando come il picco di attività tra le 19:00 e le 21:00 corrisponda alla sera europea e al pomeriggio americano, mentre è interessante osservare come, dopo il picco delle 20:00, il calo dei giocatori rallenti tra l'1:00 e le 4:00, associabile al fatto che questo orario coincide con il momento di maggiore attività americana, ma anche con quello di minor utilizzo europeo. Analogamente, il periodo tra le 7:00 e le 9:00 registra i valori più bassi della giornata, dal momento che combacia con la tarda mattinata europea e la tarda notte americana, entrambi periodi di minima affluenza.

1.3.3 Stazionarietà e autocorrelazione

Uno dei passaggi fondamentali nell'analisi di una serie storica riguarda la verifica della sua *stazionarietà*, condizione necessaria per l'applicazione di molti modelli classici di previsione come ARIMA. Per indagare la presenza di eventuali strutture di dipendenza e valutare il grado di stazionarietà, si ricorre comunemente all'analisi della funzione di autocorrelazione (ACF) e della funzione di autocorrelazione parziale (PACF). I rispettivi grafici, riferiti alla serie giornaliera dei giocatori attivi, sono riportati in Figura 1.4.

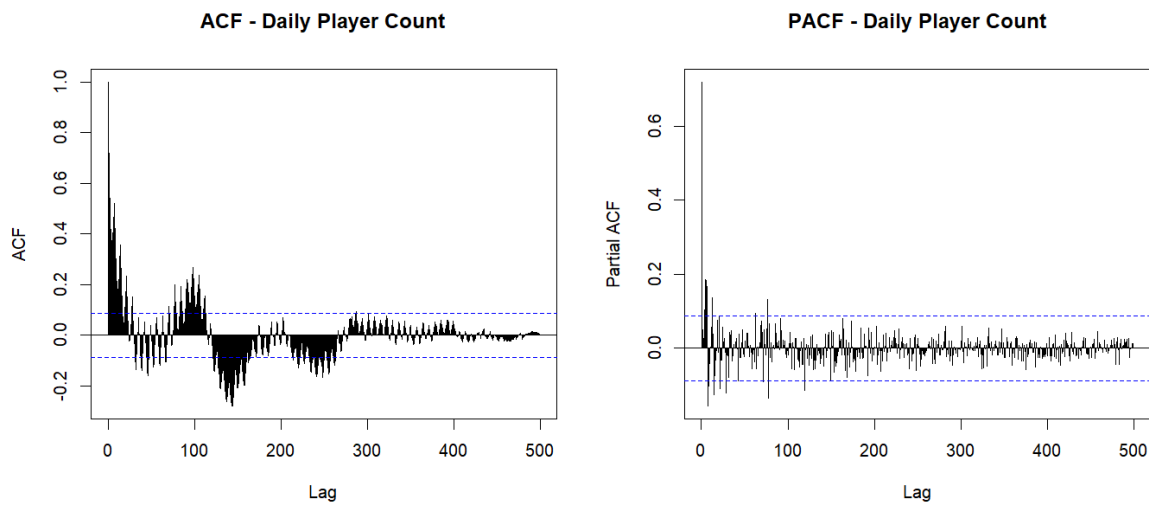


FIGURA 1.4: Funzioni ACF e PACF del numero di giocatori giornaliero.

L'analisi dell'ACF evidenzia una decrescita lenta ed esponenziale dell'autocorrelazione al crescere del lag. Questo comportamento suggerisce una forte dipendenza temporale tra le osservazioni e, soprattutto, la presenza di non stazionarietà nella media, causata in parte dal Covid. In altre parole, la serie mostra un'elevata persistenza dei valori passati, con effetti che si propagano anche a distanza di molti giorni.

Il grafico della PACF, invece, mostra valori che si mantengono entro le bande di confidenza per quasi tutti i lag successivi. Questo comportamento è tipico di un processo autoregressivo, dove la dipendenza diretta si concentra sul primo ritardo, mentre le correlazioni residue risultano trascurabili.

Nel complesso, la combinazione dei due grafici, e in particolare il decadimento lento dell'ACF, suggerisce che la serie necessiterà di una differenziazione per essere resa stazionaria nella modellazione con ARIMA.

Capitolo 2

Il modello Prophet

L'analisi delle serie temporali sul numero di giocatori attivi in un videogioco è complessa perché diversi fattori si intrecciano tra loro. Ci sono innanzitutto i trend a lungo termine, che raccontano l'evoluzione della popolarità del titolo: una fase iniziale di crescita, seguita magari da un periodo di stabilizzazione o, al contrario, da un lento declino. A questo andamento generale si sovrappongono le dinamiche ricorrenti, come l'aumento di giocatori nei fine settimana, durante le vacanze estive o nel periodo natalizio, che riflettono abitudini e momenti di maggiore disponibilità di tempo libero. Accanto a questi elementi più regolari, si verificano poi cambiamenti improvvisi legati a fattori esterni, come l'uscita di aggiornamenti, eventi promozionali o nuovi contenuti, che possono determinare picchi o variazioni inattese nell'attività dei giocatori.

Prophet è un modello reso disponibile e mantenuto dai Prophet Contributors (2025) che nasce proprio per gestire scenari di questo tipo, progettato specificamente per prevedere serie temporali con trend a lungo termine, stagionalità ricorrente e picchi dovuti a eventi esterni. Le informazioni presentate in questo capitolo si basano principalmente sul lavoro di Taylor & Letham (2017), in cui viene descritta la metodologia, l'approccio additivo e le componenti principali del modello.

L'idea fondamentale di Prophet è quella di rappresentare la serie temporale come una somma di componenti additive, ciascuna delle quali cattura un aspetto diverso del comportamento dei dati. In particolare, la serie viene scomposta come:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t),$$

dove $g(t)$ è la componente di trend, $s(t)$ descrive le componenti stagionali periodiche, $h(t)$ rappresenta l'effetto di festività o eventi particolari e $\varepsilon(t)$ è il rumore residuo.

In questo modo ogni componente è modellata in maniera separata, consentendo all'analista di capire e gestire meglio i fattori che determinano l'andamento complessivo della serie, rendendo questa formulazione non solo utile per la previsione, ma anche per l'interpretazione.

2.1 Verosimiglianza e prior: l'obiettivo del fitting

Formalizzando il modello, indicando con $X(t)$ la matrice delle basi per la stagionalità (es. serie di Fourier) e con $Z(t)$ la matrice degli indicatori di eventi e festività, si ottiene la seguente forma di modello statistico nella versione con verosimiglianza gaussiana:

$$y \mid \theta \sim \mathcal{N}(g(t; k, m, \boldsymbol{\delta}, \boldsymbol{\gamma}) + X\boldsymbol{\beta} + Z\boldsymbol{\kappa}, \sigma^2 I),$$

dove i parametri del modello sono

$$\theta = \{k, m, \boldsymbol{\delta}, \boldsymbol{\gamma}, \boldsymbol{\beta}, \boldsymbol{\kappa}, \sigma\}.$$

Per evitare che il modello si adatti eccessivamente al rumore, Prophet impone dei prior sui parametri, che corrispondono a specifiche forme di regolarizzazione:

$$\boldsymbol{\beta} \sim \mathcal{N}(0, \sigma_{\beta}^2 I) \quad \text{stagionalità, penalità } \ell_2,$$

$$\boldsymbol{\kappa} \sim \mathcal{N}(0, \sigma_{\kappa}^2 I) \quad \text{festività/eventi, penalità } \ell_2,$$

$$\delta_j \sim \text{Laplace}(0, \tau) \quad \text{cambi di pendenza nei changepoints, penalità } \ell_1.$$

Il prior normale, applicato alle stagionalità e festività, equivale a una penalizzazione di tipo Ridge, la quale attenua i coefficienti ma tende a mantenerli diversi da zero, favorendo così effetti regolari e continui, come nel caso di fenomeni attesi e ricorrenti quali le stagionalità e le festività. Il prior Laplace, invece, corrisponde a una penalità Lasso, permettendo così ai coefficienti di essere ridotti a zero, invece di essere solo ridotti. Questa caratteristica è ideale per i changepoints, poiché ci si aspetta che i cambi nel trend emergano solo quando sono supportati da concrete variazioni nei dati,

come l'uscita di una grande espansione o un cambiamento drastico nel gioco, evitando che il modello interpreti delle semplici oscillazioni casuali come dei cambi strutturali, in questo modo rimane la possibilità di cambiare il tasso di crescita ma viene mantenuta la parsimonia.

Il fitting in Prophet viene eseguito tramite la libreria **Stan**, che utilizza l'algoritmo L-BFGS per stimare i parametri mediante la stima del massimo a posteriori (MAP). In pratica, il MAP consiste nel cercare il punto

$$\hat{\theta} = \arg \max_{\theta} p(\theta | y),$$

ossia i valori dei parametri che massimizzano la distribuzione a posteriori.

Per il teorema di Bayes, la posterior è proporzionale al prodotto tra verosimiglianza e prior, si arriva quindi ad avere:

$$p(\theta | y) \propto p(y | \theta) p(\theta).$$

Mentre la stima di massima verosimiglianza cerca i parametri θ che rendono i dati osservati più probabili, il MAP cerca i parametri che rendono probabile la combinazione di dati e conoscenza a priori $p(\theta)$ sul problema, permettendo così di combinare i dati osservati con la conoscenza pregressa o con assunzioni desiderate sul modello. In termini computazionali, massimizzare la posterior equivale a minimizzare il negativo del log-posterior, che nel caso di Prophet si traduce in un bilanciamento tra errore di previsione e penalizzazioni derivate dai prior Stanford University, Dipartimento di Informatica (2012).

La funzione obiettivo risultante è una combinazione di un termine di errore quadratico, derivato dalla verosimiglianza con distribuzione gaussiana, da penalizzazioni ℓ_2 sui coefficienti di stagionalità ed eventi (prior Normali) e da penalizzazioni ℓ_1 sui cambi di trend (prior Laplace). Il risultato è un modello robusto e interpretabile, in grado di catturare le dinamiche fondamentali del numero di giocatori attivi, comprendendo crescita, declino, variazioni stagionali ed effetti di eventi speciali, senza sacrificare la stabilità delle previsioni.

Dopo questa descrizione sul funzionamento generale del modello e sul processo di fitting, nei seguenti paragrafi ci si concentrerà più nel dettaglio sulle singole componenti di questo modello, il trend $g(t)$, la stagionalità $s(t)$ e l'effetto delle festività $h(t)$,

dandone sia una descrizione teorica che una dimostrazione pratica di come siano state implementate nel modello di questa analisi.

2.2 Trend lineare piecewise

Riguardo la scelta del modello del trend, Prophet propone due possibili formulazioni: una crescita logistica con saturazione e un modello lineare piecewise (a tratti) con changepoints che permettono al tasso di crescita di variare in corrispondenza di date specifiche. Poiché il numero di giocatori attivi non è vincolato da una soglia massima nota, ma invece osserva cambiamenti improvvisi di direzione, come picchi o cali rapidi dopo una festività o un evento, la scelta ottimale è sfruttare il **trend lineare con changepoints**.

Il modello quindi si esprime come:

$$g(t) = (k + \mathbf{a}(t)^\top \boldsymbol{\delta})t + (m + \mathbf{a}(t)^\top \boldsymbol{\gamma}),$$

dove k è la pendenza iniziale (tasso di crescita/decrecita), m è l'intercetta iniziale, $\mathbf{a}(t) \in \{0, 1\}^S$ è un vettore indicatore che si attiva dopo ciascun changepoint s_j , $\boldsymbol{\delta}$ rappresenta le variazioni di pendenza nei changepoints e $\boldsymbol{\gamma}$ sono termini di correzione per garantire la continuità del livello.

In pratica, ad ogni changepoint, il modello può modificare la pendenza della retta che descrive il trend, permettendo di rappresentare scenari come un rapido aumento di popolarità dopo il lancio del gioco, o dopo l'avvenimento del Covid nel caso di questa analisi, oppure a rallentamenti e accelerazioni in corrispondenza di aggiornamenti ed eventi importanti. Va notato anche come il meccanismo di regolarizzazione spinga molte variazioni δ_j a valori vicini a zero, mantenendo quindi solo pochi cambi effettivi, evitando così che il modello si adatti in modo eccessivo al rumore.

Per costruire gli intervalli di incertezza del trend, Prophet adotta un approccio via simulazione, in cui a partire dai cambi osservati nella pendenza del trend, viene stimata una scala tipica λ dei salti δ_j . Sulla base di questa stima si generano possibili scenari futuri, campionando cambi di pendenza che rispettano la frequenza media dei changepoint storici. In questo modo l'incertezza del trend non si limita a riflettere il solo errore

residuo, ma incorpora anche la possibilità che in futuro si verifichino nuovi cambiamenti di tendenza simili a quelli osservati in passato.

Tuttavia, questa strategia poggia sul forte presupposto che la frequenza e l'ampiezza del trend futuro saranno le stesse mantenute nel passato. In presenza di shock strutturali improvvisi, questa assunzione rischia di non essere realistica e può ridurre la capacità del modello di anticipare correttamente un aumento dell'incertezza. Come infatti si vedrà nell'analisi, questa ipotesi risulterà limitante a causa dell'impatto causato dalla pandemia, poiché il modello non rifletterà un aumento dell'incertezza associata al trend, anche quando è evidente nei dati osservati.

2.3 Stagionalità, eventi e festività

Oltre al trend, un videogioco mostra pattern ricorrenti legati al comportamento dei giocatori. Si possono notare cicli settimanali, ad esempio un aumento degli accessi durante i weekend, cicli annuali legati a periodi particolari come le vacanze natalizie o estive, e, nei dati con maggiore granularità, cicli quotidiani che riflettono le abitudini di gioco della vita di tutti i giorni.

Prophet modella queste componenti tramite una base di Fourier, che permette di approssimare funzioni periodiche con precisione crescente in base all'ordine (N) scelto:

$$s(t) = X(t)\boldsymbol{\beta} = \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right).$$

La regolarizzazione sui coefficienti (a_n, b_n) assicura che la stagionalità stimata sia liscia e non venga distorta da oscillazioni spurie. Per esempio, con $P = 7$ giorni possiamo catturare la ciclicità settimanale, con $P = 365.25$ quella annuale. L'analista può quindi specificare uno o più periodi stagionali, ciascuno con un numero variabile di armoniche, in base alla granularità e alla complessità del segnale. Nel caso di questa analisi sono stati mantenuti gli ordini di Fourier dati di default, dal momento che la loro variazione non ha portato a miglioramenti o cambiamenti significativi.

Oltre alla stagionalità regolare, Prophet permette di includere eventi discreti o intervalli anomali che influenzano temporaneamente la serie, come festività, promozioni o eventi speciali all'interno del videogioco. Questi elementi sono introdotti nel modello

come regressori binari, che assumono valore 1 nei giorni in cui l'evento si verifica e 0 altrimenti.

La componente risultante è:

$$h(t) = Z(t)\kappa = [1(t \in D_1); \dots; 1(t \in D_L)]\kappa,$$

dove $Z(t)$ è la matrice degli indicatori degli eventi e κ i coefficienti che ne rappresentano l'impatto medio. In questo modo il modello può adattarsi isolando i picchi dovuti a ricorrenze prevedibili, senza attribuirli impropriamente al trend o alla stagionalità.

Nel contesto di questa analisi, le festività incluse nel modello sono state Halloween, Ringraziamento, Natale, Capodanno e San Valentino. Queste date sono state selezionate sulla base dell'analisi esplorativa precedente, poiché risultano avere l'impatto maggiore sul numero di giocatori attivi. Grazie a questa scelta, Prophet sarà in grado di riconoscere tali eventi e di adattare le previsioni quando si ripresenteranno negli anni successivi, migliorando così la capacità del modello di catturare queste variazioni stagionali prevedibili.

Analogamente, sono stati inclusi come regressori i principali eventi legati al gioco, capaci di influenzare temporaneamente il numero di giocatori attivi. Tra questi, si considerano i primi quattro *Rocket Pass*, pass stagionali a pagamento che introducono contenuti aggiuntivi, il terzo e il quinto anniversario del gioco, l'evento di Halloween del 2019 e l'evento del Capodanno Lunare cinese del 2020. L'inclusione di tali eventi consente al modello di cogliere variazioni eccezionali non spiegabili dalla sola stagionalità regolare. Entrando nei panni dell'azienda, tutti questi eventi sono stati considerati come programmati; quindi, anche se i dati utilizzati per una previsione non arrivano alla data d'inizio effettivo di questi eventi, si assume che l'azienda sia a conoscenza della loro presenza e voglia gestirne l'impatto.

Un caso particolare, invece, è rappresentato dalla pandemia di COVID-19. Trattandosi di un evento imprevedibile e senza precedenti, esso è stato incluso nel modello solo fino all'ultimo giorno con osservazioni disponibili. In questo modo, Prophet non apprenderà il nuovo trend spropositato causato dal COVID, evitando così che le previsioni vengano troppo distorte da questo evento eccezionale.

Dalla Figura 2.1 si osserva chiaramente come il modello scomponga la serie nelle sue principali componenti. Il trend mostra una crescita graduale della base utenti dall'inizio

alla fine del 2019, seguita da un'accelerazione più marcata nei primi mesi del 2020. Le festività evidenziano variazioni improvvise e negative in corrispondenza dei giorni selezionati, confermando l'importanza di includerle per spiegare i cali temporanei dei giocatori attivi. La stagionalità settimanale mostra invece un andamento regolare, con il minimo di partecipazione il martedì e un progressivo aumento verso il weekend, in linea con le abitudini di gioco. La stagionalità annuale mette in evidenza ricorrenze ricorrenti, come i cali primaverili e i picchi in corrispondenza dei periodi festivi di fine anno. Infine, i regressori esogeni evidenziano il forte impatto dei principali eventi legati al gioco (Rocket Pass, anniversari, eventi tematici), che producono cambiamenti temporanei ma significativi rispetto al livello di base della serie.

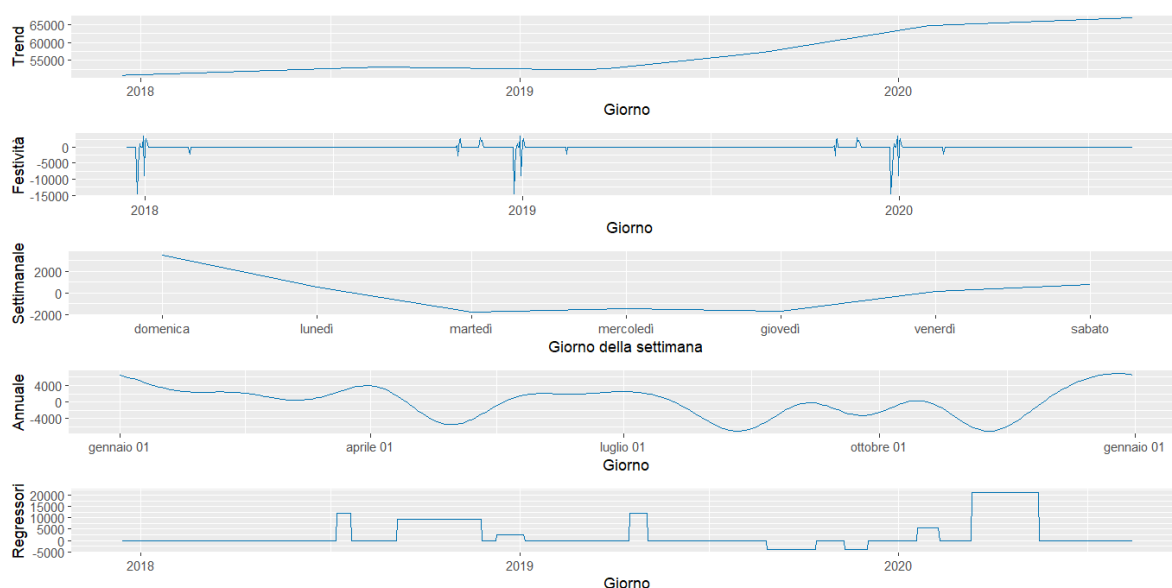


FIGURA 2.1: Componenti della serie storica.

Nel complesso, queste componenti mostrano come Prophet riesca a separare efficacemente l'effetto delle dinamiche regolari, come trend e stagionalità, da quello degli eventi più occasionali e straordinari, fornendo una base interpretativa solida per il confronto delle previsioni con quelle di ARIMA, svolte nel prossimo capitolo.

Capitolo 3

Previsione e confronto tra Prophet e ARIMA

Dopo le analisi preliminari e l'introduzione di Prophet nei capitoli precedenti, questo capitolo si concentra sul valutare e comparare le prestazioni di Prophet rispetto al modello ARIMA, introdotto da Box & Jenkins (1970) e utilizzato come baseline. Al fine di indagare la capacità dei modelli di adattarsi a contesti e scale temporali differenti, il confronto viene condotto tramite cross-validation, variando sia la granularità temporale dei dati sia l'orizzonte di previsione. Inoltre, per ciascuno scenario, verranno analizzati separatamente i periodi prima e dopo lo scoppio della pandemia di Covid, in modo tale da osservare come variano le stime nei due contesti e valutare di conseguenza in che modo i modelli riescano ad adattarsi a un cambiamento improvviso di tale portata. Gli scenari che verranno analizzati nei prossimi paragrafi sono i seguenti:

1. **Previsione oraria:** questo scenario si concentra sulla capacità dei modelli di catturare dinamiche su dati ad alta frequenza e di breve periodo, utilizzando dati con granularità di 5 minuti e svolgendo le previsioni su un orizzonte di 1 ora.
2. **Previsione settimanale:** rappresenta lo scenario intermedio, tipico di applicazioni operative di medio termine, verranno utilizzati dati con un'osservazione ogni ora e un orizzonte di previsione di 2 settimane.
3. **Previsione mensile:** in questo scenario di lungo termine, l'orizzonte sarà impostato a 2 mesi di distanza, sfruttando una granularità dei dati giornaliera, con l'obiettivo di concentrarsi più sulle capacità di prevedere il trend e i pattern stagionali.

Prima di passare alle analisi, è giusto introdurre il metodo di cross-validation applicato alle serie storiche. Infatti, il metodo tradizionale, utilizzato in contesti non relativi a serie storiche, non è direttamente applicabile alle serie storiche, in quanto la permutazione casuale dei dati violerebbe l'ordine cronologico della serie e introdurrebbe informazioni future nei dati di training, compromettendo la validità delle stime. Per questo motivo, nell'ambito delle serie storiche si ricorre a una versione adattata della cross-validation.

Il principio fondamentale, come spiegato nel lavoro di Bergmeir & Benítez (2012), consiste nel costruire una serie di sottoinsiemi temporali progressivi, in cui viene innanzitutto definito un periodo iniziale di training, a partire dal quale il modello viene addestrato, per poi svolgere la previsione con questi dati sull'orizzonte definito. A questo punto, il training set viene ampliato, includendo dati più recenti e si ripete la previsione sui dati fino a questo nuovo momento. Come mostrato in Figura 3.1, questo processo viene poi ripetuto lungo tutta la serie, permettendo di stimare le prestazioni del modello rispettando l'ordine cronologico dei dati.

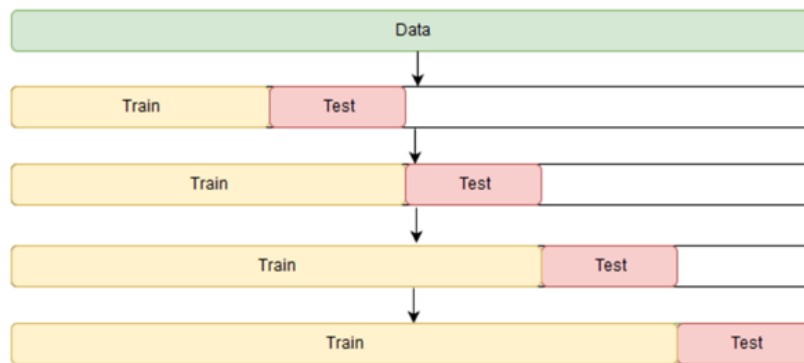


FIGURA 3.1: Esempio di cross-validation per serie temporali. Fonte: Shrivastava (2020).

In questo metodo, i punti in cui la serie viene interrotta per separare la parte di training da quella di test vengono chiamati *cutoff*, ognuno dei quali definisce un *fold*, cioè l'intervallo di tempo su cui il modello viene testato. Ripetendo il processo lungo tutta la serie, ogni fold rappresenta un'istantanea della capacità predittiva del modello, permettendo di valutare la sua stabilità e robustezza in diversi scenari temporali.

Questo permette di valutare le performance del modello in condizioni simili a quelle operative, fornendo stime più realistiche degli errori, sfruttando metriche di accuratezza come RMSE e MAPE, calcolate su ciascun fold e successivamente aggregate, così da ottenere una misura complessiva delle prestazioni del modello lungo tutta la serie storica.

3.1 Previsione Oraria

Il primo scenario analizzato utilizza il dataset con la granularità originale di 5 minuti, con l'obiettivo di prevedere l'andamento nell'ora successiva al training set. Questa tipologia di previsione riproduce una situazione operativa reale in cui è necessario stimare in tempo quasi reale il numero ottimale di server da mantenere attivi per poter garantire un'esperienza di gioco fluida, rispondendo rapidamente ai cambiamenti improvvisi nel numero di giocatori attivi.

In altre parole, la rapidità delle previsioni non solo migliora la qualità del servizio, ma consente anche una gestione dinamica e adattiva delle risorse in funzione delle variazioni del carico di gioco. Per queste ragioni, un modello che impiega troppo tempo per generare le previsioni, anche se molto preciso, rischierebbe di essere poco utile in questo contesto. Per bilanciare accuratezza e rapidità, si vuole mantenere il tempo di esecuzione inferiore ai pochi minuti, o nell'ordine dei secondi se la previsione si incentra su previsioni nell'immediato futuro. Il periodo di addestramento è stato quindi impostato tra un intervallo di 2 e 3 mesi. In questo modo, il modello riesce a catturare pattern giornalieri e settimanali abbastanza stabili da fornire previsioni affidabili con tempi di calcolo rapidi.

3.1.1 Pre-Covid

In questo primo scenario, l'analisi è stata condotta sul periodo dal 1° maggio 2019 fino al 31 luglio 2019, selezionando quindi le settimane caratterizzate da picchi di traffico tipici della stagione estiva. La cross-validation è stata eseguita con 5 fold settimanali, dove ogni cutoff dista 7 giorni dal precedente, partendo dal 3 luglio 2019 alle 20:55 e terminando il 31 luglio 2019 alle 20:55, dove ogni fold prevede un orizzonte di previsione di 1 ora, ovvero 12 intervalli da 5 minuti, arrivando così ad avere 64 giorni di training per la prima previsione e 7 in più per ogni nuovo fold. In questo modo, mantenere i cutoff distanziati esattamente di una settimana consente di valutare le performance del modello in condizioni comparabili, dato che ciascun fold copre lo stesso ciclo settimanale.

Nel grafico in Figura 3.2 si osserva come Prophet riesca a catturare meglio il trend complessivo in tutti e 5 i casi, con più difficoltà nei primi passi della previsione del 17 luglio, mentre ARIMA presenta errori più pronunciati, soprattutto man mano che ci si allontana dal cutoff.

Il secondo grafico in Figura 3.3 riporta le metriche MAPE e RMSE in funzione dell'orizzonte di previsione, includendo anche la media complessiva, rappresentata dalle linee tratteggiate. Da queste metriche si vede come ARIMA ottenga performance in media leggermente migliori nei primissimi passi di previsione (5 – 10 minuti), con MAPE intorno allo 0.009 – 0.018. Prophet, pur partendo con errori leggermente superiori nei primi minuti, mantiene prestazioni più stabili e decisamente migliori nelle previsioni più lontane, mantenendo un MAPE medio di 0.017 e RMSE medio di 1065 contro 0.043 e 2421.9 di ARIMA.

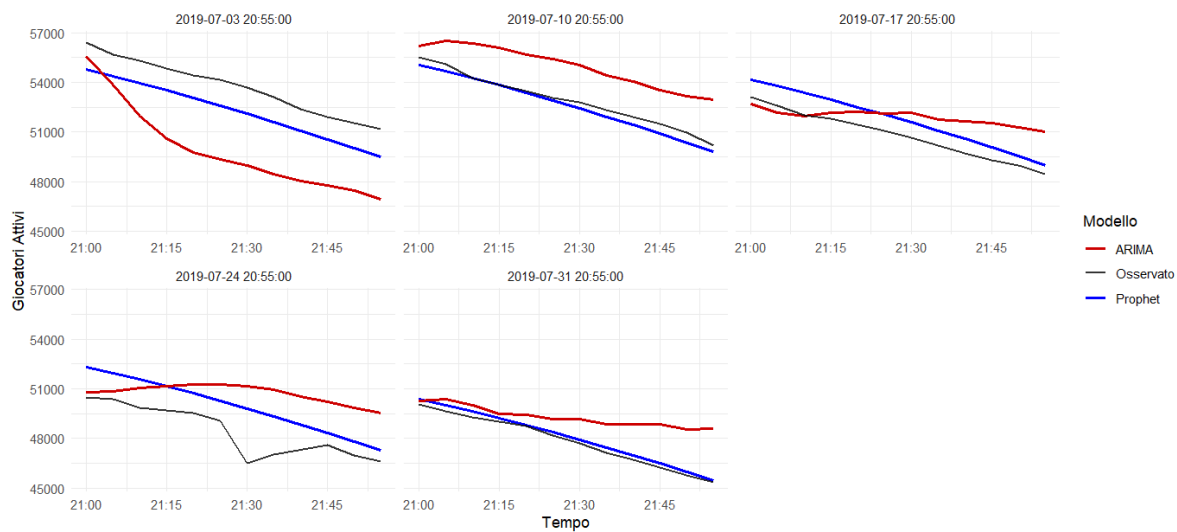


FIGURA 3.2: Confronto della previsione della cross-validation (1h).

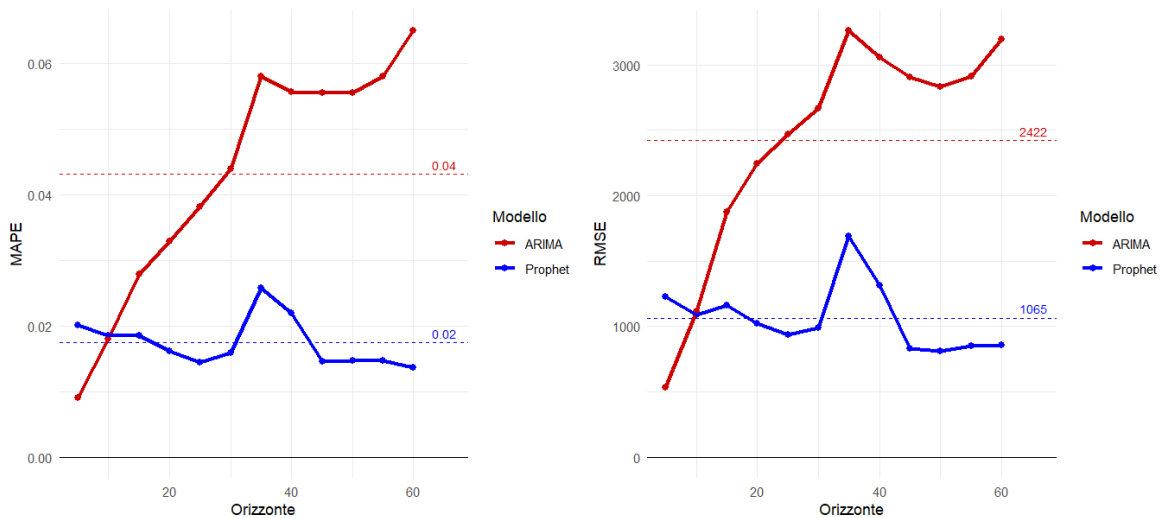


FIGURA 3.3: Confronto di MAPE ed RMSE per orizzonte (1h).

Guardando invece più nello specifico i singoli fold in Tabella 3.1, si nota come Prophet produca errori nettamente più bassi in tutti i 5 cutoff. In 2 in particolare sembra che entrambi i modelli faticino di più nel prevedere la serie, in particolare nel primo fold, Prophet ha sovrastimato sistematicamente su tutta la previsione, seppur mantenendo l'andamento corretto, a contrario di ARIMA che ha sottostimato notevolmente nel primo tratto, per poi riuscire a stabilizzarsi. Il quarto fold invece presenta un calo inaspettato di quasi 3000 giocatori in pochi minuti, che ha causato un aumento degli errori in entrambi i modelli.

Cutoff	Prophet	ARIMA
2019-07-03 20:55:00	1440	3808
2019-07-10 20:55:00	326	2045
2019-07-17 20:55:00	946	1272
2019-07-24 20:55:00	1536	2300
2019-07-31 20:55:00	248	1468

TABELLA 3.1: Errori medi per cutoff (1h).

3.1.2 Post-Covid

Per il periodo successivo allo scoppio della pandemia, l'analisi è stata effettuata negli stessi giorni, dal 1° maggio 22:00 al 31 luglio 22:00, ma dell'anno successivo, ovvero il 2020, mantenendo lo stesso orario per la previsione (21:00 - 22:00) e la stessa struttura di validazione incrociata utilizzata per il pre-Covid con 5 cutoff settimanali, orizzonte di previsione pari a 1 ora e training iniziale di circa due mesi.

In questo contesto, dalla Figura 3.4 emerge un quadro simile al precedente dove Prophet tende a seguire meglio il trend discendente osservato, anche se con una leggera sottostima in alcune situazioni, mentre ARIMA mostra prestazioni peggiorate rispetto a prima, mostrando una tendenza a sovrastimare i dati. Questo comportamento deriva probabilmente dall'aspettativa che il picco serale non sia ancora stato raggiunto e che quindi i valori debbano continuare a crescere, potenzialmente dovuto in parte alla maggiore irregolarità introdotta dalle variazioni durante il periodo pandemico, che non sembra influenzare particolarmente Prophet.

Nel secondo grafico, in Figura 3.5, i risultati di MAPE e RMSE mostrano che ARIMA, nonostante la sovrastima, ottiene ancora buone performance nei primi intervalli tra i 5 e

20 minuti, con MAPE intorno a 0.005–0.015, ma degrada rapidamente, arrivando a 0.071 e RMSE superiore a 4400 all’orizzonte dei 60 minuti. Prophet, pur partendo con errori più alti nei primi minuti, in modo simile al caso pre-Covid, mantiene un andamento molto più stabile e controllato, con valori medi di $\text{MAPE} = 0.024$ e $\text{RMSE} = 1963.6$, nettamente inferiori al MAPE ed RMSE di ARIMA, rispettivamente di 0.039 e 2558.2.

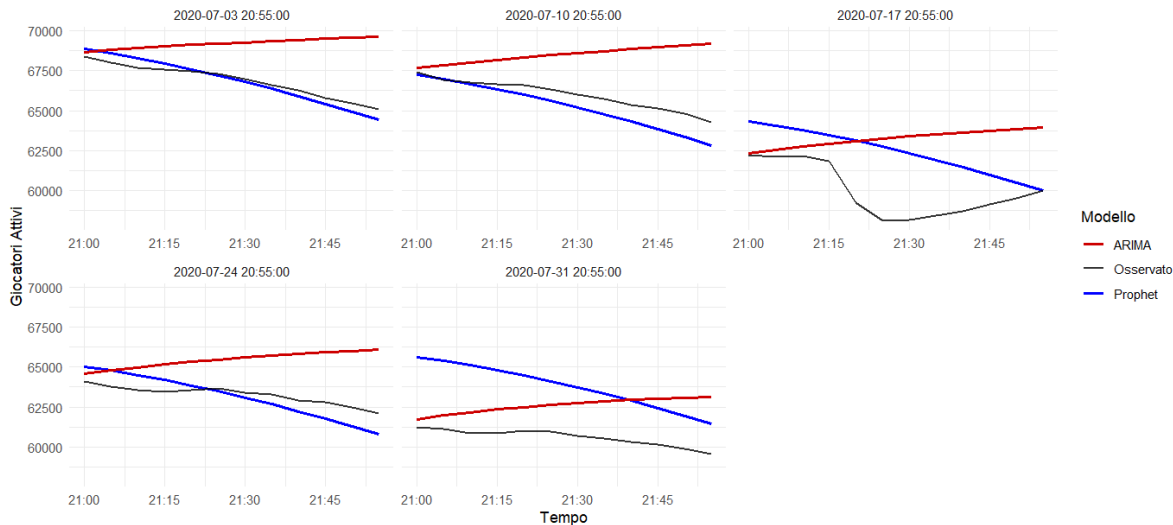


FIGURA 3.4: Confronto della previsione della cross-validation (1h con Covid).

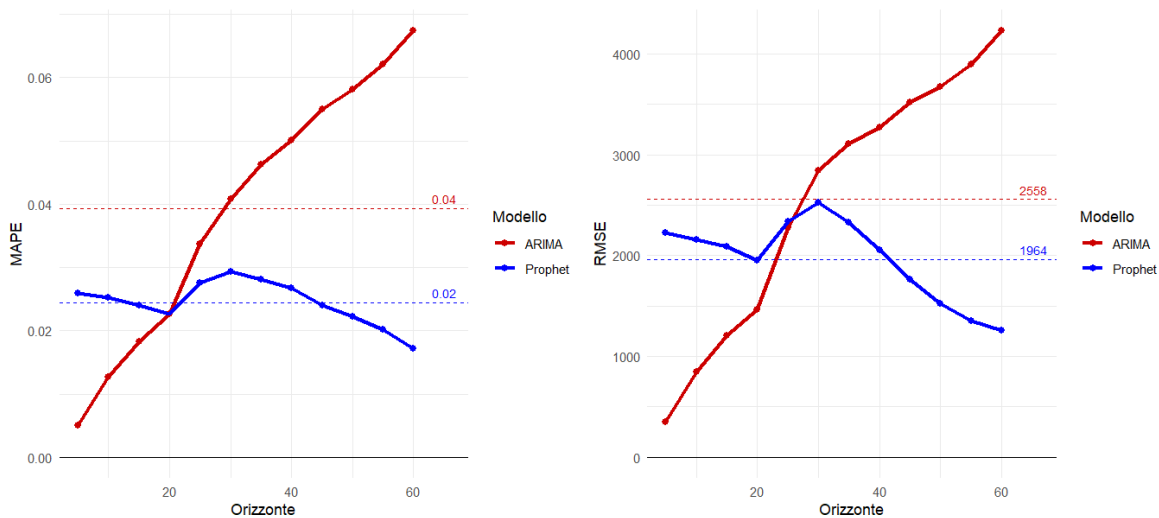


FIGURA 3.5: Confronto di MAPE ed RMSE per orizzonte (1h con Covid).

Come prima, guardando agli errori nei singoli fold in Tabella 3.2 si presenta una situazione simile, in cui Prophet risulta il migliore per la maggior parte, ma 2 previsioni mettono in maggiore difficoltà i 2 modelli. In particolare l’ultimo fold mostra ARIMA svolgere una previsione migliore, dove invece Prophet sovrastima nettamente il calo dei valori.

Cutoff	Prophet	ARIMA
2019-07-03 20:55:00	384	2324
2019-07-10 20:55:00	751	2498
2019-07-17 20:55:00	2418	3276
2019-07-24 20:55:00	764	2204
2019-07-31 20:55:00	3165	1992

TABELLA 3.2: Errori medi per cutoff (1h con Covid).

Dall'analisi delle previsioni orarie emerge quindi che Prophet si conferma più robusto in presenza di condizioni di domanda instabile, come quelle tipiche del periodo pandemico. ARIMA, al contrario, evidenzia un pattern sistematico di errore, segnale di una minore capacità di adattarsi ai cambiamenti di comportamento della serie. L'effetto della pandemia si riflette comunque chiaramente sui risultati, portando entrambi i modelli a registrare un aumento dell'errore medio rispetto al periodo pre-Covid, a testimonianza della maggiore imprevedibilità della domanda. Tuttavia, in questo caso Prophet riesce a contenere meglio l'impatto, mantenendo una maggiore stabilità anche in questo scenario più irregolare.

Un aspetto particolarmente rilevante riguarda anche i tempi di esecuzione. Nel caso pre-Covid, la procedura di cross-validation ha richiesto circa 2 minuti per Prophet e poco più di 3 minuti per ARIMA, evidenziando un leggero vantaggio per Prophet anche in termini di rapidità. Nel periodo post-Covid i ruoli vengono invertiti con che Prophet ha impiegato 1 minuto e 37 secondi, mentre ARIMA solo 49 secondi. Come si noterà dalle prossime analisi, questo sarà l'unico scenario in cui ARIMA risulta sensibilmente più veloce, ma vale comunque la pena segnalarlo, mostrando che non sempre Prophet è necessariamente più veloce.

In definitiva, per previsioni ad alta granularità e con orizzonti di breve termine, Prophet si dimostra complessivamente il modello più adatto. La sua capacità di adattarsi meglio a variazioni improvvise e a una domanda instabile lo rende preferibile rispetto ad ARIMA, che soffre invece di una maggiore rigidità e di errori più sistematici, giustificando pienamente la sua scelta come modello di riferimento in questo contesto. Se però l'obiettivo diventa una previsione immediata in tempo reale, ARIMA sembra la scelta ottimale tra i 2 modelli grazie alle migliori previsioni negli intervalli immediatamente successivi alla fine dei dati, in cui invece Prophet fatica maggiormente.

3.2 Previsione Settimanale

Il terzo scenario analizzato prende in considerazione dati aggregati su base giornaliera, con l'obiettivo di stimare l'andamento dei successivi due mesi. A differenza degli scenari a breve e medio termine, qui la previsione assume un ruolo prettamente strategico: non si tratta di rispondere a fluttuazioni immediate della domanda o di programmare le risorse nel giro di giorni, ma di avere una visione più ampia sull'andamento complessivo dei giocatori.

Previsioni su un orizzonte mensile consentono di individuare tendenze strutturali e pattern stagionali che sfuggono a una granularità più fine, risultando particolarmente utili per attività di pianificazione strategica, decisioni di marketing. Per esempio, anticipare l'arrivo di un periodo di maggiore traffico può permettere di predisporre in anticipo nuove campagne pubblicitarie mirate o di organizzare potenziamenti infrastrutturali con sufficiente margine operativo.

I tempi di calcolo del modello perdono quindi ulteriore centralità, ciò che conta maggiormente è la capacità di catturare gli andamenti in modo affidabile, anche a costo di tempi di elaborazione più lunghi, purché le previsioni restituiscano una base solida su cui costruire decisioni strategiche.

3.2.1 Pre-Covid

In questo scenario, il periodo di addestramento iniziale è stato fissato a 445 giorni, corrispondenti a ben più di un anno di osservazioni. Una finestra così ampia ha permesso di catturare la stagionalità settimanale con buona precisione, tralasciando tuttavia quella annuale, la quale aggiunta in Prophet non ha portato a miglioramenti significativi quando utilizzata su meno di due anni di dati. Per garantire anche qua un paragone sensato delle previsioni su intervalli sufficientemente diversi, la procedura di cross-validation è stata condotta con un passo di 12 settimane e un orizzonte di previsione pari a 14 giorni.

Osservando dai grafici risultanti, come si osserva nella Figura 3.6, Prophet riesce a seguire con buona fedeltà l'andamento complessivo della serie, pur mostrando oscillazioni più smorzate nei picchi e una tendenza a sovrastimare i valori. ARIMA, al contrario, in alcune previsioni coglie meglio le variazioni di breve periodo, ma tende spesso a sottostimare la serie storica. A primo impatto sembra però che ARIMA riesca a prevedere decisamente meglio l'andamento serie, ed è anche chiaramente dimostrato dall'analisi

delle metriche in Figura 3.7, dove ARIMA ottiene risultati complessivamente migliori, in particolare nelle primissime ore come nell'analisi precedente, avendo un MAPE che oscilla tra 0.009 e 0.018. Prophet, invece, parte con errori più alti nelle prime ore, ma in questo caso mantiene un errore costantemente maggiore di ARIMA su quasi tutto l'intervallo di previsione, portando Prophet a registrare un MAPE medio pari a 0.15 e un RMSE di 5511, mentre ARIMA raggiunge prestazioni migliori con MAPE di 0.1 e RMSE di 3646.

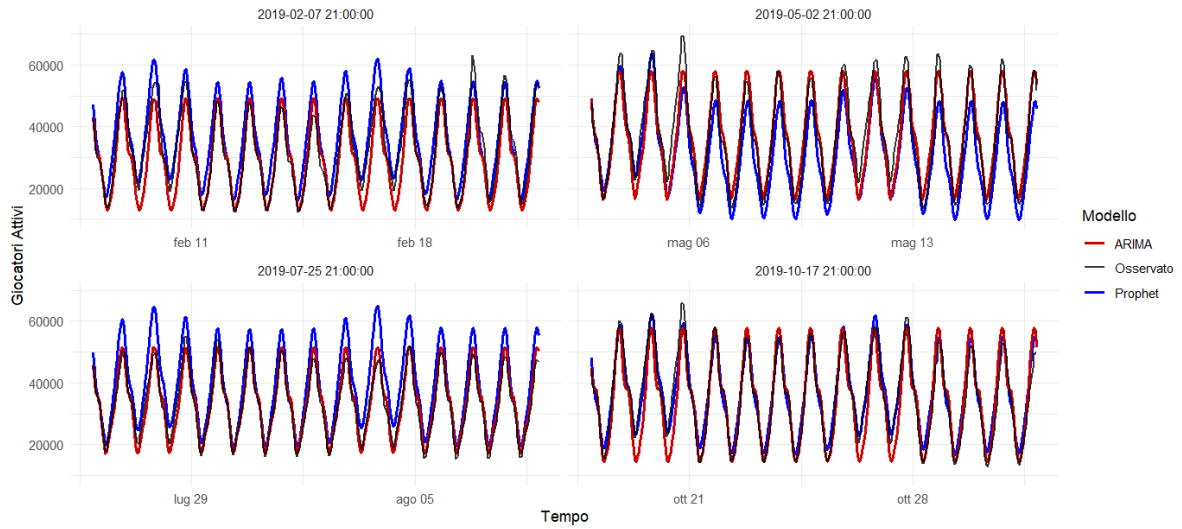


FIGURA 3.6: Confronto della previsione della cross-validation (2 settimane).

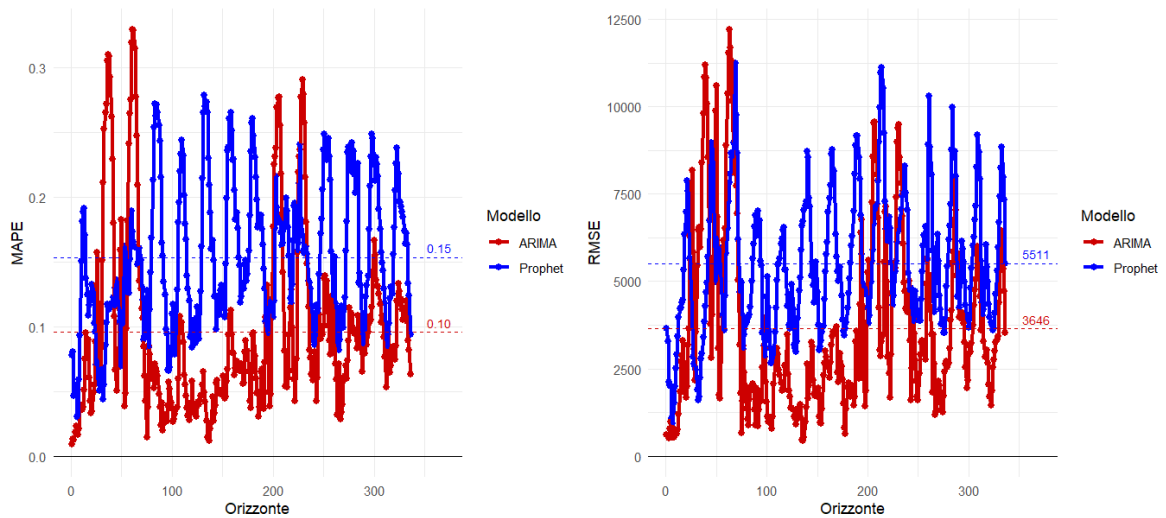


FIGURA 3.7: Confronto di MAPE ed RMSE per orizzonte (2 settimane).

Anche gli errori medi per fold in Tabella 3.3 conferma la maggiore stabilità di ARIMA in questo contesto, fatta eccezione per l'ultimo fold in cui Prophet riesce a svolgere una previsione migliore. Nonostante ciò, a differenza della previsione oraria, ARIMA riesce

a gestire meglio la maggior parte delle previsioni, dove invece Prophet mostra errori nettamente più elevati, evidenziando come non sia in grado di superare ARIMA in scenari come quello analizzato.

Cutoff	Prophet	ARIMA
2019-02-07 21:00:00	4624	3813
2019-05-02 21:00:00	6073	3690
2019-07-25 21:00:00	5838	1775
2019-10-17 21:00:00	2725	3508

TABELLA 3.3: Errori medi per cutoff (2 settimane).

3.2.2 Post-Covid

Per il periodo successivo allo scoppio della pandemia, è stato mantenuto il numero di cutoff e l'orizzonte di 2 settimane, ma questa volta l'analisi è stata svolta utilizzando tutte le osservazioni disponibili, terminando quindi con l'ultima previsione dell'ultimo fold nel 13 agosto 2020. Per poter ottenere sufficienti fold da mostrare la robustezza dei modelli, mantenendo tempi di calcoli contenuti, i cutoff distano 8 settimane l'uno dall'altro, con il primo che parte il 13 febbraio 2020, arrivando quindi ad avere un primo set di addestramento di 791 giorni e quindi, con più di 2 anni di dati, si introduce ora una stagionalità annuale di cui tenere conto.

In questo contesto, nonostante i cambiamenti, dalla Figura 3.4 emerge un quadro simile al precedente dove però risulta più evidente una delle lacune principali di Prophet, ovvero l'aspettativa che la varianza nelle oscillazioni dei dati rimanga la stessa del passato. Guardando ai primi 2 fold infatti, Prophet sottostima i picchi e sovrastima i cali, seppur seguendo correttamente l'andamento giornaliero nel periodo tra un picco e il calo successivo. Riesce invece a svolgere un lavoro migliore negli ultimi 2 fold, quando l'impatto del Covid inizia a diminuire, restando comunque inferiore rispetto ad ARIMA.

Il confronto tra le metriche MAPE e RMSE riportato in Figura 3.9 conferma questa tendenza. ARIMA mantiene prestazioni più regolari su tutto l'orizzonte di due settimane, con un MAPE medio relativamente contenuto di circa il 6% e un RMSE pari a 3423. Prophet, al contrario, parte con errori più elevati e mostra un andamento dell'errore molto più forte, caratterizzato da picchi accentuati che ne riducono l'affidabilità complessiva, comportando un MAPE dell'11% e un RMSE di 5350.

Nel complesso, i grafici confermano come in questo scenario l'ARIMA riesca a gestire con maggiore efficacia le previsioni, mentre Prophet tende a oscillare sistematicamente tra sovrastima e sottostima, senza riuscire a garantire una stabilità comparabile.

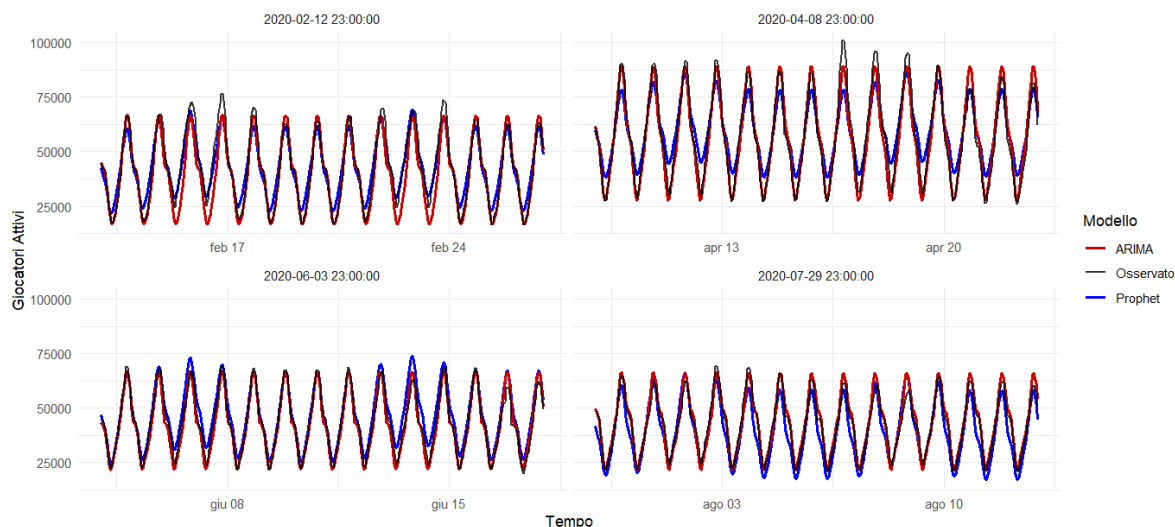


FIGURA 3.8: Confronto della previsione della cross-validation (2 settimane, periodo post-Covid).

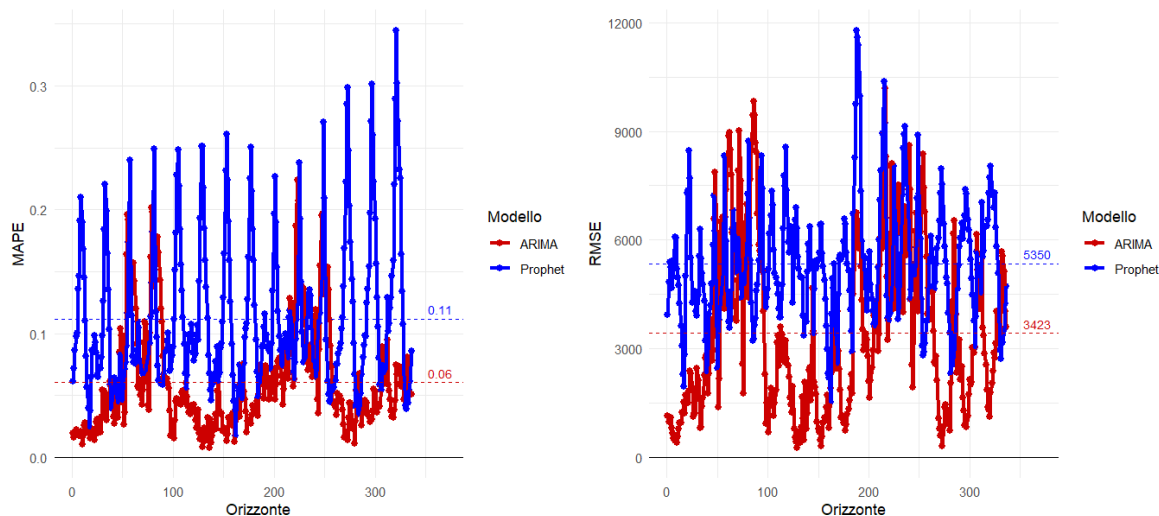


FIGURA 3.9: Confronto di MAPE ed RMSE per orizzonte (2 settimane, periodo post-Covid).

La Tabella 3.4 conferma ulteriormente la maggiore robustezza di ARIMA. In tutti i cutoff analizzati, i valori di errore risultano inferiori rispetto a quelli di Prophet, segnalando una sensibilità molto più marcata di quest'ultimo ai cambiamenti strutturali introdotti dalla pandemia. Ciò rafforza l'idea che, in contesti caratterizzati da discontinuità improvvise, un modello come ARIMA possa superare Prophet in termini di affidabilità.

Cutoff	Prophet	ARIMA
2020-02-12	3914	3685
2020-04-08	6151	3556
2020-06-03	3455	2019
2020-07-29	4538	2044

TABELLA 3.4: Errori medi per cutoff (2 settimane con Covid).

Dunque, in questo contesto ARIMA si conferma complessivamente più accurato e stabile su tutto l'orizzonte di previsione, riuscendo a gestire meglio le variazioni giornaliere anche in un contesto turbolento come quello post-Covid. Prophet, al contrario, mostra una marcata tendenza sia a sovrastimare sia a sottostimare i valori in corrispondenza dei picchi, a causa della sua assunzione che la frequenza del trend rimanga costante nel tempo, caratteristica che lo rende meno adattabile ai cambiamenti improvvisi della varianza della serie storica.

Riguardo all'effetto suscitato dalla pandemia, rispetto al periodo pre-Covid, la differenza tra i due modelli diventa più marcata. La discontinuità strutturale introdotta dal periodo post-Covid penalizza in maniera significativa Prophet, mentre ARIMA mantiene prestazioni migliori e più coerenti lungo l'intero orizzonte di previsione.

L'aspetto più positivo di Prophet riguarda i tempi di calcolo impiegati per le cross-validation che sono state completate in tempi molto più rapidi. Prophet ha infatti impiegando 45 secondi e 1 minuto e 47 secondi, rispettivamente per il caso prima e dopo il Covid, paragonato ai 5 minuti e 37 secondi per il caso pre-Covid e ai 6 minuti e 28 secondi del caso post-Covid di ARIMA. Sugerendo che Prophet potrebbe potenzialmente compensare parte delle sue lacune sfruttando quantità maggiori di dati, senza incidere in modo critico sui tempi computazionali. Essendo però che in questo tipo di scenario mantenere un tempo di calcolo particolarmente basso non è un fattore essenziale, ARIMA resta la scelta più adeguata per svolgere previsioni di questa natura.

3.3 Previsione Mensile

Il terzo scenario analizzato utilizza dati aggregati su base giornaliera, con l'obiettivo di stimare l'andamento dei successivi due mesi. A differenza delle previsioni orarie, questa tipologia di analisi non è pensata per rispondere a variazioni improvvise e in tempo reale, ma piuttosto per supportare decisioni strategiche di lungo periodo. In questo contesto, la previsione diventa uno strumento fondamentale per individuare i trend strutturali della domanda, stimare possibili picchi o cali stagionali e pianificare in modo ottimale l'allocazione delle risorse.

Le previsioni su orizzonti mensili si rivelano particolarmente utili in ambiti come il marketing e la gestione operativa di medio-lungo termine. Per esempio, poter anticipare l'arrivo di un periodo di maggiore attività permette di predisporre campagne promozionali mirate, organizzare in anticipo il dimensionamento dell'infrastruttura tecnica o pianificare attività di manutenzione nei momenti di minor traffico.

In questo scenario, la rapidità di calcolo del modello non rappresenta un fattore critico come nelle previsioni a breve termine. Ciò che conta maggiormente è la capacità del modello di catturare correttamente i pattern ricorrenti e le tendenze di fondo della serie storica, anche a costo di tempi di elaborazione più lunghi. Per questo motivo, l'attenzione è rivolta soprattutto all'accuratezza delle previsioni e alla capacità del modello di adattarsi a cambiamenti strutturali nei comportamenti degli utenti.

Il terzo e ultimo scenario analizzato prende in considerazione dati aggregati su base giornaliera, con l'obiettivo di prevedere l'andamento dei successivi due mesi. Questa tipologia di previsione consente di supportare decisioni di pianificazione strategica a lungo termine, in particolare per analisi di trend strutturali, previsioni a scopo di marketing e allocazione di risorse su orizzonti temporali più ampi.

3.3.1 Pre-Covid

In modo analogo alla previsione settimanale, il periodo di addestramento iniziale è stato fissato fino al 5 marzo 2019, arrivando così a 445 giorni su cui allenare il modello, in modo tale da riuscire a ottenere quattro previsioni di circa due mesi prima dell'inizio del periodo influenzato dalla pandemia, concludendo con l'ultima previsione dell'ultimo fold a fine ottobre 2019.

La cross-validation è stata infatti configurata con un orizzonte di previsione pari a 60 giorni, con un nuovo fold generato ogni 60 giorni, ottenendo così fold immediatamente adiacenti l'uno all'altro, a differenza delle analisi precedenti, garantendo una copertura continua dell'intero periodo in analisi.

L'analisi delle quattro previsioni, in Figura 3.10, mostra come entrambi i modelli riescano a seguire in modo soddisfacente il trend complessivo. La differenza più risaltante tra i 2 modelli è però evidente, ARIMA infatti tende a descrivere generalmente il trend, con previsioni più semplicistiche, quasi approssimabili a una linea costante negli intervalli più lontani. Prophet al contrario mantiene una ciclicità settimanale che riesce a ricalcare meglio gli effettivi picchi e cali all'interno dei mesi, seppur in media mantenga in più casi la stessa traiettoria dell'ARIMA, fatta eccezione per il secondo fold, con cutoff fissato al 3 giugno, nel quale Prophet sottostima più duramente del dovuto il calo seguente alla fine di aprile, mantenendo poi una traiettoria mediamente inferiore ai valori osservati.

In modo analogo, nel terzo fold, entrambi i modelli vengono sorpresi dal continuo abbassamento dei valori, anche dopo un'apparente stabilizzazione, portando entrambi a mancarlo completamente. Tuttavia, nel primo e quarto fold entrambi i modelli riescono a seguire bene l'andamento complessivo, con Prophet che riesce a prevedere meglio i continui picchi e cali settimanali.

Il confronto tra le metriche di errore, in Figura 3.11, evidenzia un risultato piuttosto equilibrato in cui si vede ARIMA ottenere mediamente valori leggermente migliori, con un RMSE pari a 5380 e un MAPE medio di 0.09, mentre Prophet si attesta su un RMSE di 6189 e un MAPE di 0.10. La differenza è quindi contenuta, anche se Prophet risulta penalizzato in particolare dagli shock presenti verso la fine delle previsioni.

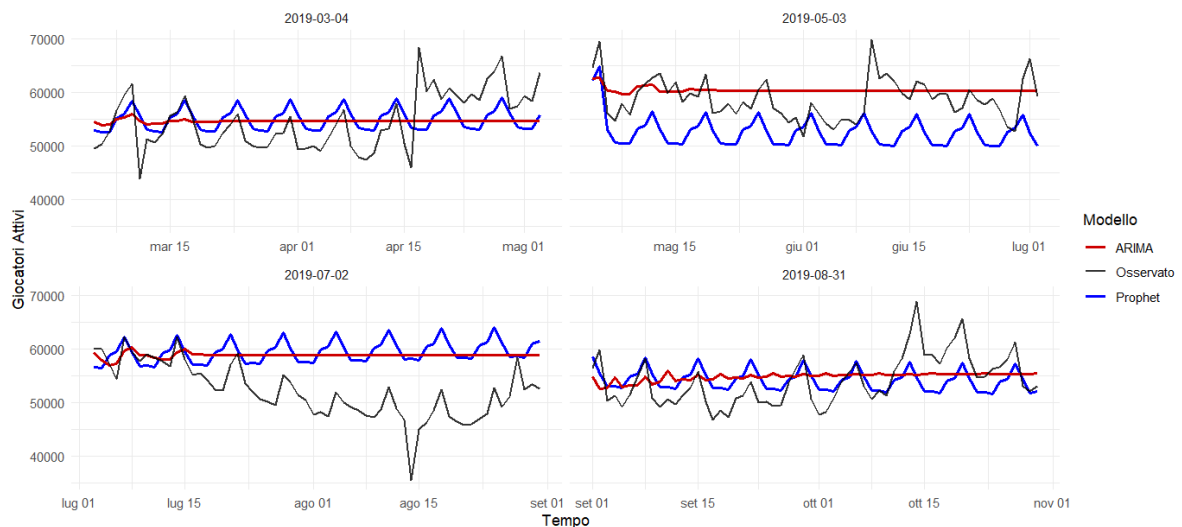


FIGURA 3.10: Confronto della previsione della cross-validation (2 mesi).

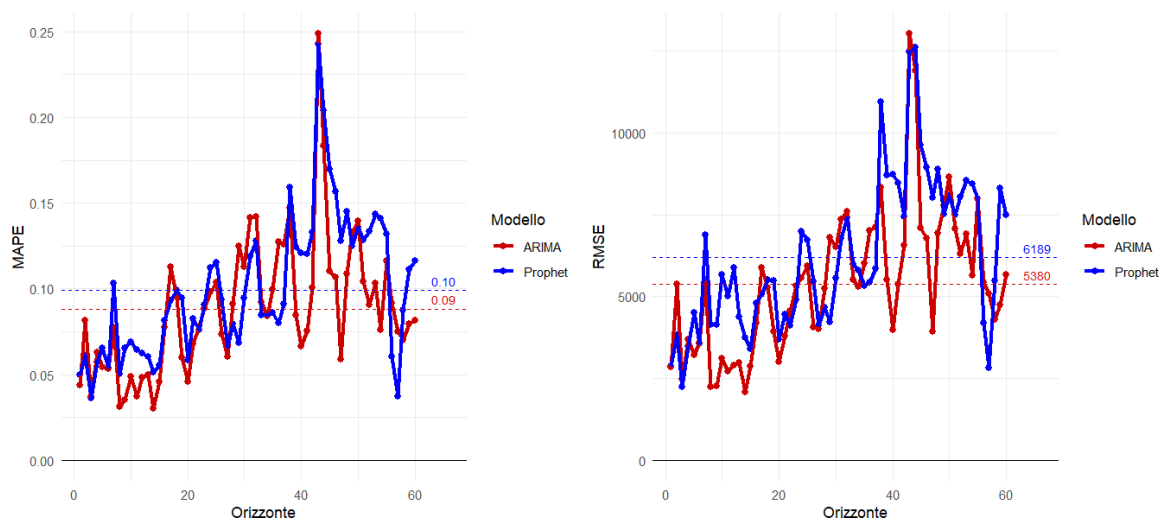


FIGURA 3.11: Confronto di MAPE ed RMSE per orizzonte (2 mesi).

La Tabella 3.5 conferma l'equilibrio tra i due modelli, mostrando come ARIMA prevalga nei 2 fold caratterizzati da un andamento più irregolare della serie, mentre Prophet, grazie al miglior adattamento alla stagionalità settimanale e alla serie che si mantiene più costante, svolge una previsione con meno errori nel primo e ultimo fold. In questo scenario, dunque, non emerge un vincitore netto, ma piuttosto una complementarità tra le due soluzioni, con Prophet più attento alla stagionalità e ARIMA più preciso nel seguire il livello medio della serie.

Cutoff	Prophet	ARIMA
2019-03-04	3951	4432
2019-05-03	6236	3275
2019-07-02	7720	6989
2019-08-31	3353	3795

TABELLA 3.5: Errori medi per cutoff (2 mesi).

3.3.2 Post-Covid

Nel periodo successivo all'inizio della pandemia, la cross-validation è stata condotta con la stessa configurazione del caso pre-Covid tramite un orizzonte di previsione pari a 60 giorni e un nuovo fold ogni 60 giorni. Mentre, come per lo scenario post-Covid della previsione settimanale, sono state sfruttate tutte le osservazioni disponibili nel dataset, arrivando ad ottenere i 4 cutoff rispettivamente nelle date 2019-12-16, 2020-02-14, 2020-04-14 e 2020-06-13.

L'analisi dei risultati mostrati in Figura 3.12 evidenzia come l'introduzione di shock improvvisi e cambiamenti strutturali nella serie renda la previsione più complessa per entrambi i modelli. In particolare, nel secondo e terzo fold, con inizio in pieno Covid rispettivamente a metà febbraio e aprile, Prophet e ARIMA seguono il trend complessivo con difficoltà. Nel primo fold invece entrambi i modelli riescono a seguire meglio i veri dati, con Prophet che risulta più performante specialmente nei primi intervalli, a contrario di ARIMA che sottostima molto più del dovuto la traiettoria dei dati. Si presenta una situazione simile nell'ultimo fold, dove ARIMA riesce a prevedere meglio la prima metà della serie, non riuscendo però ad adattarsi al calo nella seconda metà, che Prophet coglie invece molto bene.

Il confronto delle metriche nella Figura 3.13 conferma che in questo caso Prophet riesce a gestire meglio gli ultimi intervalli dei fold, riuscendo anche a seguire meglio i picchi settimanali, permettendogli di avere un MAPE ed RMSE inferiori rispetto ad ARIMA.

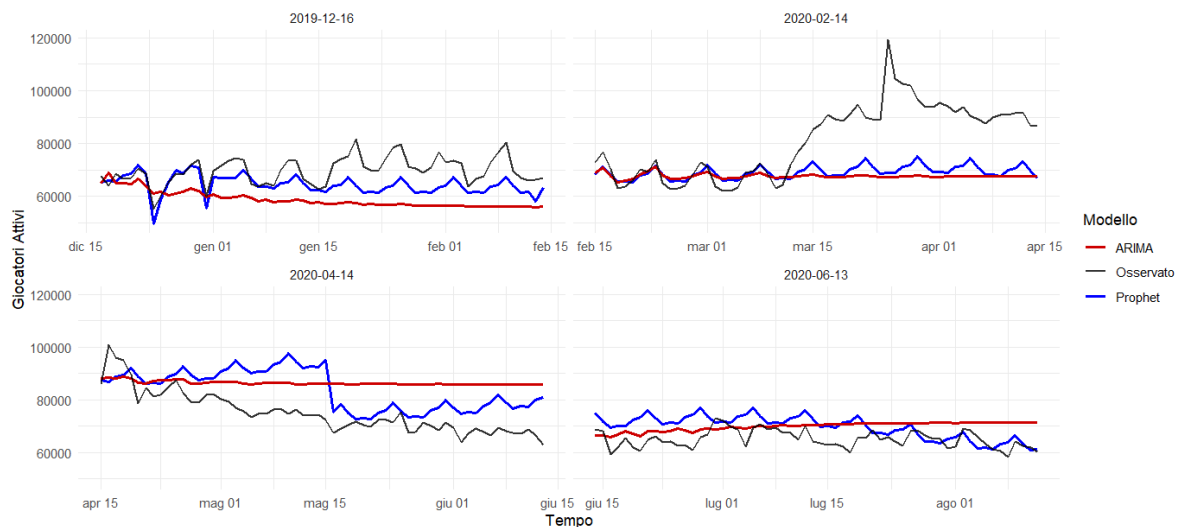


FIGURA 3.12: Confronto della previsione della cross-validation (2 mesi con Covid).

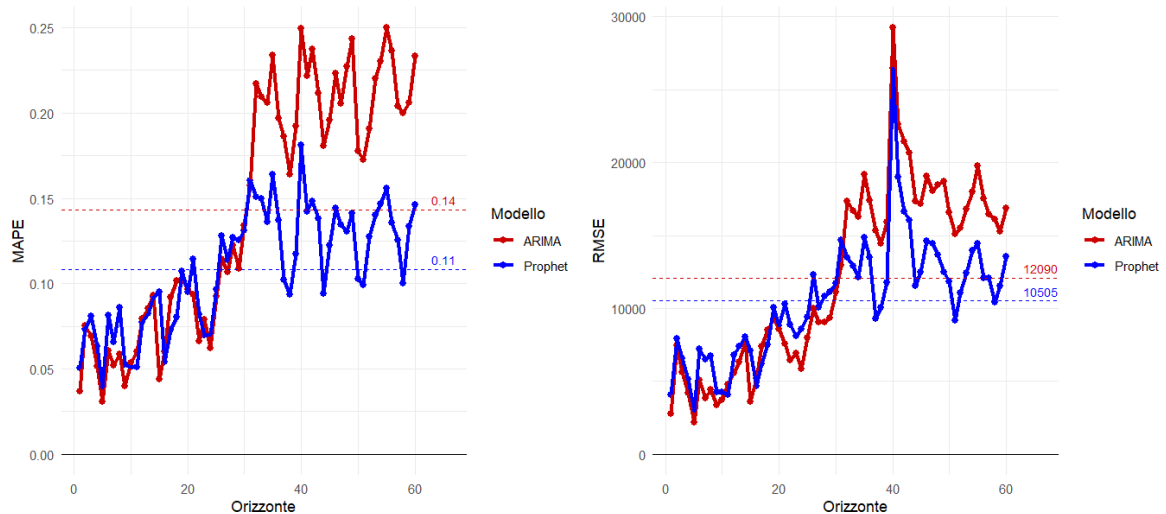


FIGURA 3.13: Confronto di MAPE ed RMSE per orizzonte (2 mesi con Covid).

La Tabella 3.6 mostra un quadro in favore di Prophet che ottiene errori simili o inferiori all'ARIMA, in particolare nel primo e terzo fold, seppur entrambi i modelli mantengano errori elevati nei fold caratterizzati dal pieno del periodo Covid.

Cutoff	Prophet	ARIMA
2019-12-16	5598	11245
2020-02-14	13047	14707
2020-04-14	9600	12253
2020-06-13	5252	5226

TABELLA 3.6: Errori medi per cutoff (2 mesi con Covid).

Concludendo quest'ultimo scenario, Prophet si conferma ottimo per cogliere la stagionalità settimanale, riuscendo inoltre ad adattarsi leggermente meglio ai cambi di livello più repentini, nonostante in alcuni casi tenda a sottostimare il livello della serie, specialmente nei primi orizzonti. ARIMA segue tutto sommato meglio il livello medio della serie, ma continua a produrre previsioni appiattite, incapaci di reagire agli shock, mostrando errori significativamente maggiori nei cutoff centrali, anche se in alcune fasi, come nel giugno 2020, fornisce risultati più comparabili a Prophet. L'aumento generalizzato dell'errore medio rispetto al periodo pre-Covid testimonia la maggiore difficoltà di entrambi i modelli nell'adattarsi a un contesto caratterizzato da forte instabilità e variazioni impreviste.

Riguardo i tempi di esecuzione delle cross-validation, entrambi i modelli risultano molto rapidi nel periodo pre-Covid, con soli 4 secondi sia per Prophet che per ARIMA. Il periodo post-Covid per Prophet resta altrettanto veloce, impiegando solo 5 secondi, mentre ARIMA richiede tempi maggiori, con 16 secondi per l'intero processo, ma comunque molto rapidi e più che ottimi nel contesto di una previsione mensile.

In conclusione, Prophet si dimostra globalmente più adatto alla previsione a due mesi su granularità giornaliera, grazie alla sua maggiore flessibilità nel seguire i cambiamenti strutturali della serie, pur con alcune sottostime. ARIMA invece risulta più stabile nel catturare la media del trend, ma appare meno reattivo in presenza dei cambiamenti nel trend.

Capitolo 4

Stima della capacità server ottimale

Nel contesto dei giochi online in tempo reale come *Rocket League*, una corretta previsione del numero di utenti attivi è solo il primo passo per garantire un'infrastruttura reattiva e stabile. Il vero obiettivo operativo è tradurre tale previsione in una stima affidabile del numero di server da mantenere attivi in una data finestra temporale, in modo da minimizzare la latenza, evitare congestioni nei tempi di matchmaking e ottimizzare i costi.

In questa sezione si propone un approccio per stimare il numero minimo di server richiesti a partire da una previsione puntuale del numero di giocatori attivi, definito nell'ambito come il numero di CCU o Concurrent Users, ottenuta mediante i 2 modelli. Per rendere l'analisi più concreta, si formulano una serie di ipotesi operative, ispirate a fonti pubbliche e pratiche comuni nel settore dei giochi multiplayer online.

4.1 Assunzioni operative

Ai fini della presente analisi, si farà riferimento alle stime puntuali ottenute dai modelli Prophet e ARIMA, con una granularità temporale di 5 minuti. In particolare, verrà considerata la previsione relativa al 17 luglio 2019 alle 21:30:00, già discussa nel capitolo precedente, che presentava una previsione equilibrata tra i 2 modelli. In questo orario, il vero valore della serie storica è 50670, mentre le previsioni di Prophet e ARIMA sono rispettivamente 51574 e 52173. Resta importante notare come la differenza tra i valori può cambiare molto a seconda di quando viene presa la previsione.

Non avendo informazioni da fonti ufficiali, svolgendo una stima grossolana basata sulle informazioni presenti su ?, si assume che ciascun server di gioco sia in grado di gestire circa 20 partite simultanee, ciascuna delle quali rappresenta una modalità *2 vs 2*, tipica di Rocket League, coinvolgendo quindi 4 giocatori a partita.

Per tenere conto di eventuali picchi imprevisti, ritardi nell'attivazione di nuove istanze o squilibri temporanei nel bilanciamento del carico, si introduce un coefficiente per gestire l'overprovisioning pari al 15%, applicato al numero base di server stimati, come spiegato da DataCore Software (2025). In questo modo, il numero effettivo di server da mantenere attivi viene calcolato moltiplicando per 1.15 il valore stimato originale, garantendo così un margine di sicurezza che consente di assorbire variazioni improvvise della domanda senza compromettere la continuità del servizio.

Infine, si precisa che, per semplicità espositiva, verranno trascurati ulteriori fattori che potrebbero contribuire a una stima più precisa, come ad esempio la volatilità del numero di giocatori in funzione della data o del giorno della settimana, il ritmo con cui i giocatori entrano o escono dal gioco, nonché parametri legati al consumo di memoria, al carico della CPU e ad altri aspetti tecnici che potrebbero influenzare le prestazioni dei server e la capacità complessiva del sistema.

4.2 Traduzione del CCU in numero di server

Sulla base delle assunzioni precedentemente definite, è possibile stimare il numero di partite simultanee, partendo dalle previsioni del numero di giocatori attivi (CCU) fornite da Prophet e ARIMA, quali 51574 e 52173 rispettivamente. Poiché nel caso della modalità di gioco *2 vs 2* ogni partita coinvolge 4 giocatori, il numero di partite simultanee sarà calcolato dividendo il CCU per 4. Applicando questo calcolo alle due previsioni e arrotondando per eccesso, si ottengono circa 12894 partite simultanee secondo Prophet e circa 13044 partite simultanee secondo ARIMA.

Considerando che ogni server può gestire fino a 20 partite simultanee, il numero minimo di server necessari, sempre arrotondato per eccesso, risulta di circa 645 server per Prophet e 653 server per ARIMA.

Infine, per tenere conto dei possibili picchi imprevisti, ritardi nell'attivazione di nuove istanze o squilibri temporanei nel bilanciamento del carico, viene applicato il coefficiente

di overprovisioning pari al 15%. In questo modo, il numero totale di server da mantenere attivi sale a circa 742 server per Prophet e 751 server per ARIMA.

Tutti questi calcoli e stime sono riportati in maniera sintetica nella Tabella 4.1, che fornisce una panoramica chiara e immediata dei valori stimati per ciascun modello.

Modello	CCU	Partite	Server	Server totali con overprovisioning
Prophet	51574	12894	645	742
ARIMA	52173	13044	653	751

TABELLA 4.1: Stima dei server necessari con coefficiente di overprovisioning del 15%.

Poiché il vero valore del numero di giocatori è stato 50670, con i medesimi calcoli si arriva a un numero di server necessari uguale a 730. Da queste operazioni si osserva come la stima basata su ARIMA porti a un numero di server leggermente superiore rispetto a Prophet, con una differenza di circa 9 server per il caso analizzato. In termini relativi, ciò equivale a un incremento dell'1% ogni intervallo di 5 minuti.

Sebbene nell'immediato tale margine possa apparire trascurabile, la sua rilevanza cresce sensibilmente se considerato su orizzonti più ampi. Una sovrastima sistematica dei server comporta infatti costi operativi aggiuntivi significativi, soprattutto in infrastrutture cloud in cui la tariffazione è proporzionale al numero di istanze attive e al tempo di utilizzo. Se assumiamo un costo orario di \$0,12 per server, valore preso da Amazon Web Services (2025), il costo operativo per l'intervallo considerato può essere stimato semplicemente moltiplicando per 0.12 il numero di server ricavato prima, ottenendo 89.04\$ e 90.12\$ all'ora rispettivamente per Prophet ed ARIMA. La differenza risultante tra i 2 è solamente di poco più di 1\$, se però estendiamo il calcolo su un giorno intero, la differenza diventa di 26\$ e, come mostrato in modo riassuntivo nella Tabella 4.2, considerando un periodo di 30 giorni questa differenza inizia ad aumentare sempre di più, arrivando ad un costo di 64110\$ per Prophet e 64890\$ per ARIMA.

Modello	Costo orario	Costo giornaliero	Costo mensile
Prophet	89.04\$	2.137\$	64110\$
ARIMA	90.12\$	2.163\$	64890\$

TABELLA 4.2: Stima dei costi operativi dei server.

Ciò si traduce in un risparmio di circa 780\$ al mese, dimostrando come anche una differenza relativamente piccola nella stima del numero di server possa avere un impatto significativo sui costi operativi mensili. Una previsione più accurata consente quindi di ottimizzare le risorse senza compromettere il margine di sicurezza necessario per gestire i picchi di domanda.

In questo senso, l'analisi evidenzia come un modello predittivo più preciso non migliori soltanto le metriche teoriche di errore, ma produca un beneficio economico concreto.

È importante osservare che la stima effettuata si basa su un dimensionamento statico per un intervallo di tempo specifico. In un ambiente reale, l'infrastruttura di *Rocket League* opererebbe probabilmente con un sistema mirato al dimensionamento dinamico, attivando o disattivando server in funzione della domanda osservata in tempo reale. In pratica, questa stima fungerebbe da baseline per una pianificazione anticipata in vista di eventi attesi, come l'inizio di un fine settimana o il lancio di una nuova stagione competitiva.

In conclusione, l'esercizio mostra come il passaggio dalla previsione della domanda alla stima della capacità server ottimale non sia affatto banale, dove anche piccole differenze nella precisione del modello predittivo possono avere un impatto consistente a livello operativo ed economico. Una valutazione completa delle performance dei modelli, che consideri non solo le metriche di errore ma anche i costi legati alla gestione anticipata delle risorse, rappresenta quindi un passaggio fondamentale per sistemi di gioco online con traffico ad alta intensità, come *Rocket League*.

Capitolo 5

Conclusioni

Il lavoro condotto in questa tesi ha mostrato come la previsione della domanda di utenti attivi nei giochi online, nello specifico in *Rocket League*, possa rappresentare un elemento chiave per la gestione efficiente delle infrastrutture server. Utilizzando il modello Prophet, è stato possibile generare previsioni su diverse granularità temporali integrandole con assunzioni operative relative alla capacità dei server e ai comportamenti di gioco.

Dall'analisi emerge che Prophet si dimostra particolarmente robusto nelle condizioni di domanda instabile, come quelle osservate durante la pandemia di COVID-19, e più veloce di ARIMA nella maggior parte degli scenari, soprattutto quando si tratta di orizzonti di previsione brevi e di elevata granularità. Questo lo rende uno strumento adatto anche per aziende di dimensioni ridotte o con personale limitato, grazie alla facilità d'uso e all'interpretabilità dei risultati, senza necessità di competenze statistiche avanzate. Tuttavia, i risultati settimanali hanno evidenziato i limiti di Prophet nel catturare variazioni di breve periodo in contesti turbolenti, dove ARIMA si è dimostrato più stabile e accurato. Analogamente, nelle previsioni mensili, Prophet ha gestito meglio gli shock improvvisi e la stagionalità settimanale, mentre ARIMA ha mantenuto una maggiore precisione nel livello medio della serie, pur reagendo meno prontamente agli eventi straordinari.

L'integrazione della previsione del numero di CCU con le assunzioni operative sul numero di partite gestibili per server e sul margine di overprovisioning ha permesso di stimare quantitativamente il fabbisogno di server. Questo approccio non solo consente di dimensionare le risorse in anticipo, ma offre anche una base metodologica per definire politiche di dimensionamento dinamico dei server e per ottimizzare i costi operativi nei

giochi multiplayer ad alto traffico. Come dimostrato nell'analisi dei costi, una previsione leggermente più accurata può tradursi in un risparmio economico concreto di centinaia di dollari al mese, evidenziando l'impatto diretto delle scelte modellistiche sulle decisioni operative.

È importante sottolineare che nel mondo reale le grandi compagnie di gaming utilizzano modelli predittivi più avanzati e complessi, con Prophet impiegato spesso solo come baseline. Tuttavia, per realtà più piccole, con minore esperienza e risorse limitate, Prophet rappresenta una soluzione pratica ed efficace, grazie alla sua flessibilità, interpretabilità e robustezza di fronte a shock imprevisti. Inoltre, questa analisi è stata condotta su *Rocket League*, un gioco molto popolare caratterizzato da numerosi eventi in-game, aggiornamenti frequenti e variazioni dovute a periodi festivi. Va tuttavia considerato che entrambi i modelli potrebbero ottenere previsioni più accurate in contesti con giochi caratterizzati da un traffico minore di giocatori, dove la serie storica dei giocatori è più regolare e ciclica. Proprio questi giochi potrebbero rappresentare il principale target per aziende interessate a utilizzare Prophet per previsioni di questa natura.

In conclusione, la tesi evidenzia come la combinazione di modelli predittivi, assunzioni operative e analisi dei costi possa supportare decisioni informate per la gestione delle infrastrutture server nei giochi online. Prophet si conferma un modello valido per scenari instabili e granularità elevate, fornendo una solida base per studi futuri e per l'implementazione di strategie di dimensionamento e ottimizzazione dei costi in contesti reali di gaming online.

Bibliografia

- AMAZON WEB SERVICES (2025). Amazon gamelift server pricing.
- BERGMEIR, C. & BENÍTEZ, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Computers in Biology and Medicine* **41**, 440–451.
- BOX, G. E. P. & JENKINS, G. M. (1970). *Time Series Analysis: Forecasting and Control*. Holden-Day. Prima edizione. Successive edizioni con Reinsel e Ljung, Wiley, 2015.
- BYTEBREW (2025). Bytebrew insights: Player play times.
- DATACORE SOFTWARE (2025). Overprovisioning explained: Maximizing capacity vs. efficiency.
- PROPHET CONTRIBUTORS (2025). Prophet: Forecasting procedure for time series.
- R CORE TEAM (2023). R: A language and environment for statistical computing. Version 4.3.1.
- SELECT DATASET (2023). Rocket league player statistics.
- SHRIVASTAVA, S. (2020). Cross validation in time series.
- STANFORD UNIVERSITY, DIPARTIMENTO DI INFORMATICA (2012). Chapter 7.5: Maximum a posteriori estimation (da *probability statistics with applications to computing* di alex tsun).
- TAYLOR, S. J. & LETHAM, B. (2017). Forecasting at scale. PeerJ Preprints 5:e3190v2.
- VALVE CORPORATION (2025). Steam platform. Dati originali dei giocatori.

